

Constantin Shoot Vayevoda

Securing a Distributed, Self-Organising IoT
Sensing Fleet:

Mutual Certificate-Based DTLS 1.3,
Revocation, and Autonomous Recovery

Treball de Fi de Grau

dirigit per Edgar Batista de Frutos
i Antoni Martínez Ballesté

Grau d'Enginyeria de Sistemes i Serveis de Telecomunicacions



UNIVERSITAT ROVIRA i VIRGILI

Tarragona

2026

Abstract

A fleet of salvaged-hardware noise (NMU) and air-quality (AMU) sensing units, deployed in smart environments for active ageing, carries its telemetry over mutual, certificate-based DTLS 1.3 (RFC 9147): every device and the server hold a CA-signed identity, sessions are forward-secret, and a compromised or retired device is revoked from a dashboard and refused on its very next transmission, independently of the cryptographic session. Around that secure channel, the system adds autonomous per-tier recovery (a ladder of increasingly costly remedies, timed from last confirmed contact rather than socket state), zero-touch discovery so a server address change never bricks a unit, database retention and a daily statistics push to a research endpoint, and an AI operator layer (Model Context Protocol) restricted to a fixed, closed set of deterministic tools, so that prompt injection has no reachable action to trigger. The design is evaluated against a parallel final project in the same research group: a four-rung comparison on deployed hardware shows that the predecessor design’s pre-shared ECB scheme repeats its own plaintext’s block structure almost one-for-one, undetectable by entropy alone, while the system presented here is indistinguishable from random at the block level and additionally confines the theft of one unit to that unit, where the predecessor’s receiver key is carried by every device. The result is a working proof of concept, measured on real hardware with real attack reproduction, sustained at two continuously-running units and captured at sixteen, with its remaining limitations (device-key custody, certificate lifetime, and fleet-scale rollout) stated explicitly as the next phase of work.

Keywords: DTLS 1.3, mutual authentication, IoT security, autonomous fault recovery, certificate-based revocation, embedded systems

Resum

Una flota d'unitats de sensat de soroll (NMU) i de qualitat de l'aire (AMU), construïdes amb maquinari recuperat i desplegades en entorns intel·ligents per a l'envelliment actiu, transporta la seva telemetria sobre DTLS 1.3 (RFC 9147) mutu i basat en certificats: cada dispositiu i el servidor tenen una identitat signada per una autoritat de certificació, les sessions ofereixen confidencialitat persistent (*forward secrecy*) i un dispositiu compromès o retirat es revoca des d'un tauler de control i es rebutja en la seva següent transmissió, amb independència de la sessió criptogràfica. Al voltant d'aquest canal segur, el sistema afegeix recuperació autònoma per capes (una escala de remeis progressivament més costosos, temporitzats des de l'últim contacte confirmat i no des de l'estat del sòcol), descobrint sense configuració perquè un canvi d'adreça del servidor mai no inutilitzi una unitat, retenció de base de dades i un enviament diari d'estadístiques a un punt final de recerca, i una capa d'operador d'intel·ligència artificial (Model Context Protocol) restringida a un conjunt fix i tancat d'eines deterministes, de manera que una injecció d'instruccions no disposi de cap acció assolible per activar. El disseny s'avalua davant d'un treball de fi de grau paral·lel del mateix grup de recerca: una comparació de quatre graons sobre maquinari desplegat mostra que l'esquema ECB amb clau precompartida del disseny predecessor reproduceix gairebé un a un l'estructura de blocs del seu propi text pla, indetectable només amb l'entropia, mentre que el sistema que aquí es presenta és indistingible d'una font aleatòria a escala de bloc i, a més, limita el robatori d'una unitat a aquella mateixa unitat, mentre que la clau del receptor del disseny predecessor la porten tots els dispositius. El resultat és una prova de concepte operativa, mesurada sobre maquinari real amb reproducció real d'atacs, sostinguda amb dues unitats en funcionament continu i capturada amb setze, amb les limitacions que resten (custòdia de la clau de dispositiu, vida útil dels certificats i desplegament a escala de flota) indicades explícitament com la fase de treball següent.

Paraules clau: DTLS 1.3, autenticació mútua, seguretat IoT, recuperació autònoma de fallades, revocació basada en certificats, sistemes encastats

Resumen

Una flota de unidades de sensado de ruido (NMU) y de calidad del aire (AMU), construidas con hardware recuperado y desplegadas en entornos inteligentes para el envejecimiento activo, transporta su telemetría sobre DTLS 1.3 (RFC 9147) mutuo y basado en certificados: cada dispositivo y el servidor poseen una identidad firmada por una autoridad de certificación, las sesiones ofrecen confidencialidad persistente (*forward secrecy*) y un dispositivo comprometido o retirado se revoca desde un panel de control y se rechaza en su siguiente transmisión, con independencia de la sesión criptográfica. En torno a ese canal seguro, el sistema añade recuperación autónoma por capas (una escalera de remedios progresivamente más costosos, temporizados desde el último contacto confirmado y no desde el estado del socket), descubrimiento sin configuración para que un cambio de dirección del servidor nunca inutilice una unidad, retención de base de datos y un envío diario de estadísticas a un punto final de investigación, y una capa de operador de inteligencia artificial (Model Context Protocol) restringida a un conjunto fijo y cerrado de herramientas deterministas, de modo que una inyección de instrucciones no disponga de ninguna acción alcanzable que activar. El diseño se evalúa frente a un trabajo de fin de grado paralelo del mismo grupo de investigación: una comparación de cuatro peldaños sobre hardware desplegado muestra que el esquema ECB con clave precompartida del diseño predecesor reproduce casi uno a uno la estructura de bloques de su propio texto plano, indetectable solo con la entropía, mientras que el sistema aquí presentado es indistinguible de una fuente aleatoria a escala de bloque y, además, limita el robo de una unidad a esa misma unidad, mientras que la clave del receptor del diseño predecesor la llevan todos los dispositivos. El resultado es una prueba de concepto operativa, medida sobre hardware real con reproducción real de ataques, sostenida con dos unidades en funcionamiento continuo y capturada con dieciséis, con las limitaciones restantes (custodia de la clave de dispositivo, vida útil de los certificados y despliegue a escala de flota) indicadas explícitamente como la siguiente fase de trabajo.

Palabras clave: DTLS 1.3, autenticación mutua, seguridad IoT, recuperación autónoma de fallos, revocación basada en certificados, sistemas empotrados

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Dr. Antoni Martínez Ballesté and Dr. Edgar Batista de Frutos, for welcoming me into the Smart Technologies Research Group and its outstanding, diverse team. The direction of the work was set together with the team, and within that shared plan I kept the freedom to organise my own contribution toward the goals we agreed on. I am glad to have spent time there, including a period contracted as a support technician, and to have built this second phase on the experience and the skills gained in the first, a genuinely rewarding process of refining, innovating, and watching a bench prototype become a working system. Their guidance and confidence have shaped me as an engineer and as a person, and I carry with me everything my years at Universitat Rovira i Virgili have given me. I leave hopeful, looking forward to the professional path ahead and to continuing my studies at another institution.

I am deeply thankful to my parents and my family, to my friends, and to Haru, for the patience, encouragement, and steady support that carried me through this journey, without which I would not have seen it through, and to the many randomly precise encounters, events, and situations that shaped me into a new person throughout the years.

Farewell!

This work was carried out within the project *Cognitive Integration of Intelligent and Contextual Systems for Active Ageing* (PID2024-158490OB-C31), funded by MICI-U/AEI/10.13039/501100011033.

Contents

List of Acronyms	x
1 Introduction	1
1.1 Motivation	1
1.2 Problem	2
1.3 Objectives	2
1.4 Scope and limitations	3
1.5 Document structure	3
2 Background	3
2.1 Security objectives	3
2.2 Transport: why DTLS rather than TLS	4
2.3 DTLS 1.2 versus DTLS 1.3, and why the version matters here	5
2.4 AEAD, forward secrecy, and the cipher suite actually used	5
2.5 Public-key infrastructure versus a pre-shared key	6
2.6 Constrained-device and real-time constraints	6
2.7 Prior work	7
3 System design and implementation	8
3.1 Architecture overview	8
3.2 Decision 1: DTLS 1.3 over DTLS 1.2	10
3.3 Decision 2: certificate-based PKI over a pre-shared key	10
3.4 Decision 3: the negotiated cipher suite, not a chosen one	11
3.5 Decision 4: session reuse with a jittered age ceiling	11
3.6 Decision 5: revocation as a hot-reloaded name check, not a certificate- validity mechanism	12
3.7 Decision 6: UDP/DTLS over TCP/TLS for the recovery design	13
3.8 Decision 7: an authenticated server time source rather than network time	13
3.9 Decision 8: an AI operator restricted to a closed tool set	14
3.10 Decision 9: name-based discovery, with the handshake as the trust boundary	15
3.11 Decision 10: a dedicated reverse proxy for the operator plane	15
3.12 The operational mandate features	16
3.13 Preserved functionality	17
4 Security analysis	17
4.1 The network attacker	17
4.2 The physical attacker	18
4.3 The operator and AI-assistant tier	19
4.4 Catastrophic case: CA private key theft	19
4.5 Summary	20
5 Evaluation methodology	20
5.1 The evidence standard	20
5.2 Experiment design	21

5.3	What the automated gate does and does not prove	21
5.4	Capture provenance	21
5.5	Deployment scale	22
5.6	Scope of the evidence	22
6	Results	24
6.1	Four-way comparison against the predecessor design	24
6.2	Attack suite	27
6.3	DTLS 1.3 on the wire	28
6.4	Revocation, both device types	32
6.5	Autonomous recovery	34
6.6	The AI operator layer in use	35
6.7	Hardware crypto acceleration, measured and declined	37
6.8	Reliability and integrity experiments	38
7	Discussion	40
7.1	What the results establish	40
7.2	Positioning against the prior work	40
7.3	Limitations and what is not claimed	41
7.4	Availability	42
8	Future work	42
8.1	Key custody and device hardening	42
8.2	Per-certificate operator revocation	43
8.3	Zero-touch auto-enrolment	43
8.4	Scale validation	43
8.5	Firmware updates over the air	43
8.6	Event-driven sensing and a bare-framework port for the noise unit . . .	44
8.7	Post-quantum readiness	44
8.8	Performance benchmark	44
8.9	Smaller items	44
9	Conclusions	45
A	Experiment register	46
B	Phase-2 hardware changes	48
B.1	Enclosures	48
B.2	Parts removed	49
B.3	Button block, redesigned for tool-free service	49
B.4	Board fabrication and assembly	49
B.5	A constraint that shaped the firmware	50
C	MCP operator sessions	51
C.1	Status and stored-data tools	51
C.2	Live reads to the hardware	54
C.3	Deducing an unnamed alarm condition	56
C.4	A recorded model error, with the tool contract intact	57

List of Figures

Figure 1.	System overview	9
Figure 2.	The operator dashboard	10
Figure 3.	The server advertising itself over multicast DNS	15
Figure 4.	Capture file properties	22
Figure 5.	Why identical plaintext blocks matter	25
Figure 6.	The four rungs, measured on deployed hardware	25
Figure 7.	The same reading on the wire	26
Figure 8.	No identity string inside the encrypted handshake	29
Figure 9.	A hardware DTLS 1.3 handshake between AMU_16 and the deployed server	29
Figure 10.	The server’s HelloRetryRequest	30
Figure 11.	ServerHello detail for the air-quality unit (top) and the noise unit (bottom)	31
Figure 12.	Cipher-suite lists offered in the ClientHello	32
Figure 13.	Revocation of a working unit (NMU_18) from the dashboard	33
Figure 14.	The cut used for the recovery and revocation runs	34
Figure 15.	Autonomous outage recovery on NMU_22	35
Figure 16.	Fleet roll-call through MCP	36
Figure 17.	A non-trivial operator task	37
Figure 18.	Concurrent DTLS 1.3 sessions in one capture	39
Figure 19.	Enclosure, Phase-1 and the revised Phase-2 NMU casing	48
Figure 20.	Module stack, before (left) and after (right)	48
Figure 21.	Button-block module before the revision	49
Figure 22.	The assembled fleet, and assembly in progress	50
Figure 23.	nmu_status for a single noise unit	51
Figure 24.	latest_reading for AMU_15, from stored data	52
Figure 25.	A descriptive analysis over a window, in two parts	53
Figure 26.	live_read: a reading taken from the hardware	54
Figure 27.	A live read followed by a recommendation the operator can act on, in two parts	55
Figure 28.	Deducing an unnamed alarm condition, continued	56
Figure 29.	A recorded model error, with the tool contract intact	57

List of Tables

Table 1.	The network attacker against the deployed system	18
Table 2.	Scope of the evidence	23
Table 3.	One unit’s readings, four protections, one capture	24
Table 4.	Guarantees per rung	27
Table 5.	Attack suite	28
Table 6.	Hardware against software cryptography on the noise unit	38
Table 7.	Selected reliability and integrity experiments	40
Table 8.	Experiment register	46

List of Acronyms

AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
AI	Artificial Intelligence
AMU	Air-quality Monitoring Unit
API	Application Programming Interface
CA	Certificate Authority
CN	Common Name (certificate subject field)
CRL	Certificate Revocation List
DNS	Domain Name System
DTLS	Datagram Transport Layer Security
ECB	Electronic Codebook (block cipher mode)
ECDHE	Elliptic-Curve Diffie–Hellman, Ephemeral
ECDSA	Elliptic Curve Digital Signature Algorithm
ESP-IDF	Espressif IoT Development Framework
FreeRTOS	Free Real-Time Operating System
GCM	Galois/Counter Mode
HKDF	Hash-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
IoT	Internet of Things
JSON	JavaScript Object Notation
MAC	Media Access Control (network interface address)
MCP	Model Context Protocol
mDNS	Multicast Domain Name System
ML-KEM	Module-Lattice-based Key-Encapsulation Mechanism (FIPS 203)
NIST	National Institute of Standards and Technology
NMU	Noise Monitoring Unit
NTP	Network Time Protocol
OCSP	Online Certificate Status Protocol

PKI	Public Key Infrastructure
PSK	Pre-Shared Key
PSRAM	Pseudo-Static Random-Access Memory
RAM	Random-Access Memory
RFC	Request for Comments (IETF standards document)
RTC	Real-Time Clock
SHA	Secure Hash Algorithm
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
WAL	Write-Ahead Logging

1 Introduction

1.1 Motivation

An earlier phase of this same line of work built and validated two salvaged-hardware sensing kits for smart environments: a noise unit (NMU) on an ESP32 and an air-quality unit (AMU) on a Raspberry Pi. Each was shown to perform autonomous, timestamped acquisition on its own, and their combined data showed a small but consistent short-lag association between a voice-activity measure and the *rate of change* of indoor CO₂ – correlational, and modest in effect size [1]. That Phase-1 project closed by sketching a Phase-2 in which the two kits become a managed fleet; among the pieces it named were signed configuration, robust time handling, per-unit remote diagnostics, and encrypting the telemetry link. Phase 1 itself kept its data local, sending only on-demand peak summaries over Bluetooth Low Energy at a button press; the continuously reporting UDP fleet and its dashboard were built at the start of Phase 2. This work secures that link and delivers the management layer around it.

One Phase-1 capability was deliberately not carried across. That unit sampled at 44.1 kHz and ran overlapping 4096-point Fourier transforms to detect spectral activity in the voice band [1]; this one samples at 8 kHz and reports a single sound-pressure level per event, the arithmetic mean of the per-chunk decibel values over the chunks that were not silent. Because the decibel scale is logarithmic, that mean is not the event’s true energy average: it identifies how loud an event was, it does not measure it. The trade is deliberate, since the subject here is the transport rather than the acoustics, and the detection logic is isolated in one firmware module that a later phase can restore or extend without disturbing the layers below it.

A parallel Phase-2 project by Marina Rull Ventura, under the same supervisors and within the same umbrella project but on a different sensing function – the EmotiBit physiological wearable rather than an environmental unit – started from the opposite end: a device that already had a wireless telemetry link and needed it secured, which she did with a pre-shared symmetric key [2]. Her own future work names authenticated encryption with a fresh nonce per message, AES-GCM, as the step beyond that.

This project sits between the two. It takes the sensing kits of the first and the security goal of the second, and goes one step past either: not a pre-shared secret handed out in advance, but a full public-key infrastructure with mutual, certificate-based authentication, carried over Datagram Transport Layer Security version 1.3 (DTLS 1.3, RFC 9147 [3]).

This is a timely rather than a speculative choice. Comparative measurements on low-power microcontrollers show that DTLS and TLS in their 1.3 form run within the memory, bandwidth and energy limits of constrained devices; the cost is about a fifth more program memory, and in the mutually authenticated DTLS profile this system uses, fewer bytes on the air and less energy per handshake than version 1.2 rather than more [4]. What remains genuinely hard is not the protocol but the credentials it needs. A public-key infrastructure is the standard way to manage identity on the Internet, yet its supporting protocols were never designed for devices measured in kilobytes, and getting a certificate onto each unit as it is deployed is identified as the most urgent obstacle to using one at the scale the Internet of Things implies [5]. This work takes the first finding as licence to require a real handshake on every unit, and treats the second as the problem to engineer rather than to avoid.

The work is carried out within the research project *Cognitive Integration of Intelli-*

gent and Contextual Systems for Active Ageing (PID2024-158490OB-C31) [6], funded by the Spanish Ministry of Science, Innovation and Universities for 2025–2028, and developed in part within the Smart Technologies Research Group at Universitat Rovira i Virgili. That project’s aim is context-aware support for healthy, independent ageing built on artificial intelligence, the Internet of Things (IoT), and language models, with an explicitly “transparent, secure and privacy-respecting” design, to be validated in nursing homes and private residences. This work delivers the secure-transport and fleet-management component of that aim.

This is an IoT deployment in the strict sense: physically sited, mains-powered, otherwise unattended sensing nodes that report over a wireless link to a server which discovers, authenticates, configures and recovers them without a site visit. The noise unit is a constrained node as RFC 7228 [7] uses the term, that document grading such devices by how little RAM and flash they carry. It sits near the top of that scale, and the binding limit in practice is the few kilobytes of RAM a DTLS handshake needs at once, which on the no-PSRAM noise-unit tier must share internal memory with the sampling buffers.

1.2 Problem

Three problems compound on top of each other in this deployment context.

No confidentiality or authentication existed on the wire. Telemetry travelled as cleartext JSON over UDP; any party on the network could read every sensor value, and any party could inject a forged reading – there was no mechanism to verify that a packet claiming to be a given device actually originated from it.

The devices are constrained, and unevenly so. The NMU is a microcontroller running FreeRTOS under a hard real-time watchdog, with no general-purpose operating system it could restart without rebooting the whole chip. The AMU is a general-purpose Linux machine. Neither carries a battery-backed clock: the microcontroller keeps time in an on-chip low-power domain that survives a warm reset but not a loss of power, while the Raspberry Pi has no clock hardware at all and loses the time on every reboot. Any security design had to work within both machines at once, not only the more capable one.

The system had to keep working on its own. A senior-residence deployment cannot depend on a technician being present when a Wi-Fi access point drops, a server reboots, or a unit is power-cycled by an unrelated building event. Self-recovery, not just correctness under laboratory conditions, was a stated requirement from the outset.

1.3 Objectives

1. Replace the plaintext transport with true mutual authentication and forward-secret encryption, based on a real handshake and per-device certificates – not a pre-shared symmetric key.
2. Preserve and extend the original system’s functionality (discovery, buffering, the live dashboard) rather than replacing it silently; security is layered on top of a working system, never a rewrite of it.
3. Deliver autonomous recovery from realistic field faults – dropped packets, lost sessions, Wi-Fi outages, power cuts, server restarts, a rogue server – without a technician present.

4. Add the operational features required by the second-phase mandate: database retention under unbounded growth, a daily statistics push to a research endpoint, and an AI operator layer (MCP) that can query the fleet without giving an attacker a new reachable action.
5. Measure the result against Marina Rull Ventura’s parallel security project, on the evidence standard she set: each security property shown as an observed artefact – a packet capture, a device log, a database query – with the mechanism first off and then on, rather than asserted from a test that passed.

1.4 Scope and limitations

The project was proposed in February 2026 as a distributed coordination protocol, in which devices would decide their own operational role from current fleet conditions. Successive mandate revisions through 2026 changed that. Building on the Phase-1 project’s own closing recommendation, the scope settled on the security and fleet-management layer a distributed deployment needs first, and peer role election is not part of it.

What “self-organisation” means in the delivered system is therefore concrete: zero-touch server discovery, unattended fault recovery, and server-governed configuration carried over the same secure channel as the telemetry. Role election among peers, named in the February proposal, is outside this scope by decision, not deferred.

The sensing platform itself – the sensors, the acquisition firmware, the enclosures, and the air-quality/noise correlation result – is the subject of the Phase-1 project and its repository [1] and is not re-derived here; this document takes it as given and adds the transport security and fleet-operation layer on top. Appendix B records only the mechanical changes made for the Phase-2 fleet.

The system is evaluated as a working proof of concept. Sustained, continuously-running operation is demonstrated at two units (one NMU, one AMU); a larger sixteen-unit fleet is exercised for concurrency, discovery, power-cut and revocation behaviour but not soaked over a long duration. Both limits are stated explicitly in Section 7.

1.5 Document structure

Section 2 reviews the cryptographic and systems background needed to justify the design choices that follow, including the directly relevant prior work this project builds on and departs from. Section 3 presents the architecture and the ten design decisions that carry the security argument, each with the alternative considered and the evidence behind the choice made. Section 4 gives the threat model. Section 5 states the evaluation method. Section 6 reports the measured results, led by a direct four-way comparison against the predecessor security design. Section 7 discusses the results and states the system’s limitations plainly. Section 8 scopes what remains, and Section 9 closes with what was delivered against the original mandate.

2 Background

2.1 Security objectives

A security design is measured against three properties: confidentiality (no unauthorised party learns the data), integrity (no unauthorised party alters it undetected), and

availability (the system keeps working). This work addresses the first two in full: every record is encrypted and authenticated end to end, and identity is bound to a certificate rather than asserted in the payload, so a forged, replayed, or tampered record is rejected (Section 6). Availability is addressed only in the narrow sense of autonomous recovery from transient faults, such as a dropped link, a power cut, or a change of server address, and not as resistance to a deliberate denial-of-service attack, which Section 4 states out of scope. Authenticity, often listed as a fourth property, is here the mechanism the others rest on: both peers present a certificate and verify it at every handshake.

2.2 Transport: why DTLS rather than TLS

Transport Layer Security (TLS) protects a byte stream carried over TCP. Datagram TLS (DTLS) is the corresponding protocol for connectionless, message-oriented transport over UDP, and the choice between the two is a transport decision independent of anything cryptographic. A TLS/TCP stack runs perfectly well on the microcontroller used here; the reasons for choosing UDP are specific to this system’s traffic and its recovery design, not a platform limitation.

The traffic is a stream of small, independent, individually acknowledged records, not a bulk transfer: a few hundred bytes every tens of seconds, each self-contained. This system adds its own application-layer acknowledgement over that – a record carries a unique identifier, the sender waits for the server to echo it, and re-sends the same identifier on a timeout, with a uniqueness index on the server absorbing any duplicate (Section 3). That identifier is formed on both device types as a per-boot random number joined to a counter that increments once per record. It is an application-layer label with no relationship to the transport underneath it: it is what makes a resent record recognisable as the same record, and it is carried inside the encrypted payload rather than being any field of the protocol. Reliable, in-order delivery is therefore provided where it is needed, in one direction, by the application. Four properties of UDP matter for the rest of the design.

Failure detection is the application’s decision. When a datagram goes unanswered, the sender decides how long to wait before treating the server as unreachable and escalating. Over TCP, a peer that disappears silently – common on shared Wi-Fi, where no close is sent – is reported dead only after the stack exhausts its own retransmission schedule, whose backoff constants are set when the stack is built, not per connection. The recovery ladder of Section 3.7 times its escalation from the device’s own heartbeat interval, and that requires the timing to belong to the application.

No head-of-line blocking between records. A lost TCP segment withholds every later byte from the application until it is recovered, even bytes that already arrived. Independent datagrams let the recovery ladder discard a session, re-run discovery, or resume a buffered flush from where it stopped, with nothing trapped in a kernel buffer.

Connection state is the application’s to bound. One UDP socket receives every first datagram; the listener then opens a connected socket and a thread for that peer, so per-peer cost is chosen rather than imposed by the kernel. There is no accept backlog and no lingering close state to wait out. What a simultaneous fleet reconnection costs is therefore a bounded number of in-flight handshakes – capped explicitly, with peers over the cap dropped to retransmit – and a caretaker thread reclaims any that never complete (Section 3).

An address change is not a broken connection. A TCP connection is bound to a fixed source and destination address pair and breaks when either end’s address changes – a

DHCP lease renewal, a move between access points. On UDP the next datagram simply arrives from the new address: the device finds the server again by name (Section 3.10), and the server records each device’s address from the source of its authenticated records, so neither side stores an address that a network change could invalidate. Service discovery itself uses multicast DNS, which is defined only over UDP [8]: multicast has no connection-oriented transport.

2.3 DTLS 1.2 versus DTLS 1.3, and why the version matters here

DTLS 1.2 (RFC 6347 [9]) sends the certificate exchange – the part of the handshake that names the two peers – in the clear. A passive observer with no ability to break any cryptography can still see, at every handshake, which named device is talking to the server.

DTLS 1.3 (RFC 9147 [3]), built on the TLS 1.3 handshake design (RFC 8446 [10]), derives the handshake traffic keys *before* either side’s certificate is sent, so the certificate exchange itself is encrypted. The same protocol upgrade also removes the class of legacy, downgrade-prone options TLS 1.2 still permitted – static RSA key exchange, CBC-mode ciphers, compression, renegotiation – by defining only five cipher suites, all Authenticated Encryption with Associated Data (AEAD) constructions [10].

The cost of moving from version 1.2 to version 1.3 on constrained hardware has been measured directly on low-power microcontrollers, and which way the result goes depends on which resource is counted [4]. Program memory grows by roughly a fifth, which is unsurprising, since a newer protocol version means more code compiled in. Every other measured quantity moves the other way in the mutually authenticated DTLS profile closest to the one used here: for DTLS with ECDHE-ECDSA the published figures report fewer bytes on the air under 1.3 than under 1.2 (1500 against 1726 for the 128-bit configuration), lower handshake energy, and no growth in peak heap. The saving is attributed to the redesigned record layer, whose length and sequence-number fields are variable in width and in part omissible [3].

Those measurements were taken on Arm Cortex-M boards over a different radio stack, so their absolute values do not transfer to the hardware used here, and this document does not reuse them as its own results. What transfers is the sign of the difference and the reason for it: bytes on the air are a property of the protocol encoding, not of the processor that produces them, and the authors report no significant code-size difference between the boards they tested. Their measurements also excluded hardware cryptographic acceleration. The microcontroller used here provides it, and the build shipped here does not use it either, which is a decision taken on this project’s own measurements rather than an omission (Section 3 and Section 6.7). Requiring version 1.3 on the noise unit is therefore an informed engineering choice rather than an aspirational one.

2.4 AEAD, forward secrecy, and the cipher suite actually used

An AEAD cipher (here, AES-GCM [11, 12]) provides confidentiality and integrity in one pass: a single fixed-size tag authenticates both the ciphertext and any associated cleartext data, and a single flipped bit anywhere in either causes verification to fail. TLS 1.3’s cipher suite name encodes only the AEAD algorithm and the hash used for its key schedule – TLS_AES_128_GCM_SHA256, where the hash drives the HKDF key

derivation of RFC 5869 [13] – because key exchange and authentication are negotiated independently, through separate handshake extensions. Key exchange is always ephemeral Elliptic-Curve Diffie–Hellman (ECDHE), carried in the `key_share` extension: the deployed captures show the noise unit using NIST curve P-256 and the air-quality unit a hybrid P-384 plus ML-KEM-1024 group (Section 3). Authentication is ECDSA over P-256 [14, 15], carried by the certificate.

TLS_AES_128_GCM_SHA256 is the one cipher suite RFC 8446 makes mandatory for every compliant implementation; TLS_AES_256_GCM_SHA384 is only recommended. The two are not interchangeable configuration options of one suite: TLS 1.3 defines no suite that pairs AES-256 with SHA-256, so the only route to AES-256 also requires SHA-384 [16] to be compiled into the implementation. The 128-bit AES key is also the *matched* choice: NIST’s own key-management guidance places 128-bit symmetric strength and the 256-bit elliptic curve P-256 at the same security level [17], so the certificates and the record cipher already agree. This system’s embedded build ships SHA-384 disabled by the third-party library’s own default configuration, which is why every measured handshake on the microcontroller negotiates the 128-bit suite. Section 3 examines this choice and its evidence in full.

Ephemeral ECDHE gives *forward secrecy*: the values exchanged to derive one session’s traffic keys are generated fresh and discarded once the handshake completes, so a later compromise of a device’s long-term signing key does not expose any previously recorded session. The long-term key is used only to *sign* the handshake, never to derive it.

2.5 Public-key infrastructure versus a pre-shared key

A pre-shared key (PSK) design distributes one secret to a device before deployment and authenticates every message with it thereafter – simple, but with two structural weaknesses this project’s evaluation (Section 6) measures directly: the key itself *is* the identity, so nothing distinguishes a legitimate holder of the key from an attacker who copied it; and revoking one compromised device without disturbing every other device sharing (or derived from) the same secret is not naturally supported.

A certificate-based public-key infrastructure (PKI) instead has an offline Certificate Authority (CA) sign one X.509 certificate [18] per device and one for the server. At handshake time each side proves possession of its own private key and the peer verifies the signature against the CA it already trusts. Identity is then a signed, unforgeable statement rather than a shared secret, and revoking one device is a matter of refusing that one identity. What a public-key infrastructure costs in exchange is provisioning: its supporting protocols were written for full-sized machines, and putting a certificate onto each device as it is deployed is described in the literature as the most urgent obstacle to running one at the scale the Internet of Things implies [5]. This project pays that cost by hand, one unit at a time, and Section 8 scopes automating it.

The standards-based revocation mechanisms, X.509 CRLs and the online status protocol OCSP [18, 19], are consulted only at handshake time; Section 3 explains why this system uses a per-record name check instead.

2.6 Constrained-device and real-time constraints

The noise unit runs on an ESP32-S3 under FreeRTOS, a preemptive real-time operating system offering independent tasks, priorities, and queues rather than best-effort

time-sharing. Two details of this environment recur through the design: a hardware watchdog will reboot the chip if any task fails to check in within a fixed window, and there is no operating-system process underneath the firmware to restart independently of the whole device – "restarting the program" and "rebooting the machine" are the same action. The air-quality unit runs as an ordinary Linux process on a Raspberry Pi, where the two are genuinely different operations with different costs. Neither machine has a battery-backed clock, and they fail differently: the microcontroller keeps time in an on-chip low-power domain that survives a warm reset but is cleared by a loss of power, while the Raspberry Pi 4B has no clock hardware at all and starts every boot with no idea of the time. Both therefore need a time-trust mechanism, and the difference between the two failure modes is what lets one of them keep a trustworthy timestamp across a self-healing reboot (Section 3).

2.7 Prior work

Two final projects from the same research group frame this work directly.

The Phase-1 project [1], my own, built and validated the air-quality and noise sensing kits this project's devices descend from. Its Future Work section named self-organisation, a lean versioned data plane, signed configuration, robust time handling, per-unit remote diagnostics, and encryption in transit and at rest as the Phase-2 scope. This work delivers the security and fleet-management strand of that list; peer role election is outside its scope (Section 1).

The parallel project by Marina Rull Ventura [2] secured an existing IoT wearable (EmotiBit) that originally transmitted in the clear. Its design is worth stating precisely, because Section 6 measures against it. Each device holds its own pre-shared key and, on the same storage, a second key belonging to the receiving application; records are encrypted with AES-128 in ECB mode and carry an HMAC-SHA-256 tag, verified before decryption so a forgery is rejected cheaply; replay is blocked by monotonic packet numbers; and the list of authorised devices lives in a central database the receiver reads when it starts. Two aspects of that work are methodologically significant here and are deliberately reused: its evaluation shows every defended attack *with security off, then on*, as paired Wireshark evidence rather than a described property; and its own Future Work names AES-GCM – authenticated encryption with a fresh nonce per message – as the natural next step beyond ECB. Section 6 extends that off-versus-on comparison to four rungs, measured on this project's own deployed hardware, to show what a PKI-based DTLS 1.3 design adds beyond that next step.

The two references are complementary rather than competing, and this document is built to take from each what it does best. The Phase-1 project supplies the hardware platform, the deployment context, and the memory structure this text follows. Marina Rull Ventura's supplies the evaluation discipline – every security property shown as paired evidence with the mechanism off and then on – and the concrete predecessor design that the four-rung comparison measures against. Neither is contradicted here: a pre-shared scheme with per-device keys was a reasonable fit for a single wearable and a single receiver, and the argument of Section 6 is only that a managed fleet, where units are added and retired and no operator can revisit each one, needs the properties a certificate infrastructure adds on top.

3 System design and implementation

3.1 Architecture overview

The system is organised in three tiers. **Devices** (NMU, AMU) sample, detect, buffer, and transmit. The **server** terminates every DTLS session, verifies identity from the certificate rather than the payload, deduplicates and stores telemetry, and serves the operator dashboard and API. The **operator** tier reaches the server only through the same mutually-authenticated channel, either as a human through the dashboard or as an AI assistant through a closed set of MCP tools (Section 3.9). Concurrency is deliberately different per tier: the NMU is dual-core FreeRTOS, with sampling and networking on separate cores joined by a queue; the AMU is a Linux process with a sampling loop and a network worker thread sharing a bounded queue; the server runs one thread per connecting peer plus a caretaker thread that reclaims stalled handshakes. All three tiers speak DTLS 1.3 through the same library, wolfSSL [20], reached through its Arduino distribution on the microcontroller [21] and its Python binding on the Raspberry Pi and the server [22]; the complete source is published [23]. Figure 1 shows how the three tiers fit together, and Figure 2 the operator view onto them.

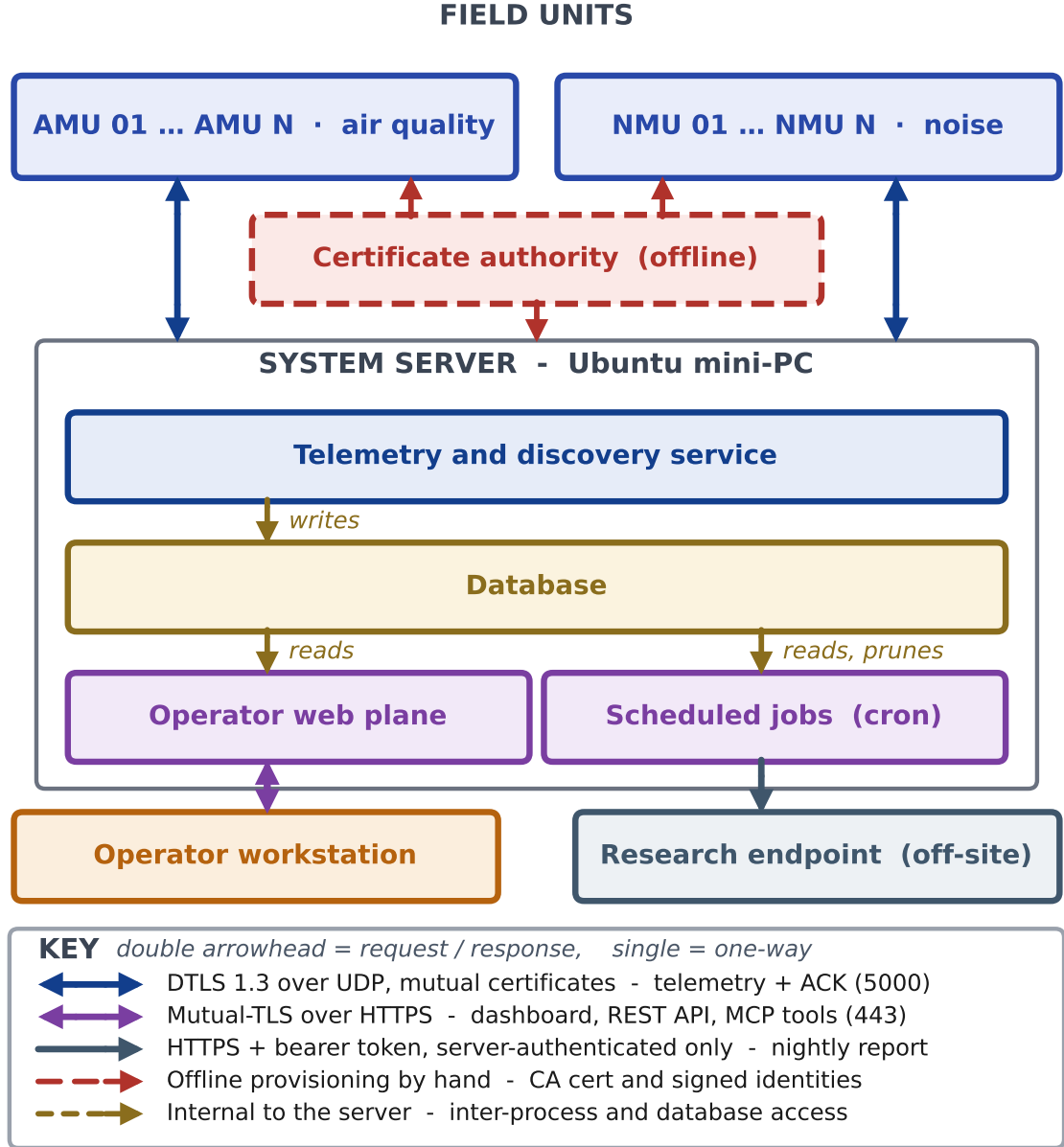


Figure 1. System overview: devices, protocols, and server software parts. Field units are Raspberry Pi 4B air-quality nodes (Python) and ESP32-S3 noise nodes (C++), each a wolfSSL DTLS 1.3 client. On the server, one process (`listener.py`, `session.py`, `discovery.py`) terminates every DTLS 1.3 handshake, takes the peer identity from the verified certificate rather than the payload, checks an allow-list and the revocation set before the payload, acknowledges, and is the only writer to a single-writer WAL SQLite database; an operator web plane (an `nginx` mutual-TLS gate in front of a Flask dashboard and JSON API bound to localhost) reads that database, as do two nightly `cron` jobs – one bounds the database by its on-disk size, the other pushes per-device statistics to the research endpoint. Every device-to-server link is a mutually authenticated DTLS 1.3 session with session reuse; the operator tier reaches the server over mutual TLS and never a device directly; the only path that leaves the local network without a client certificate is the nightly statistics report, which carries a bearer token instead. All identities are signed by one certificate authority that takes no part in the running system and is never contacted at handshake time; certificates travel to each unit by hand. Where the authority’s own private key is held, and what that costs, is stated in Section 4.



Figure 2. The operator dashboard, reached by the `smartageing.local` name the server publishes over mDNS. The two live charts and the event log are retained from the Phase-1 system; the *Device Keys* panel (per-unit revoke) and the *Device Heartbeat* panel (interval delivered on the next authenticated acknowledgement, no restart) are Phase-2 additions that drive the revocation and heartbeat-distribution experiments of Section 6.

The subsections below present the design as a sequence of decisions, each with the alternative that was considered, the evidence behind the choice made, and its acknowledged cost.

3.2 Decision 1: DTLS 1.3 over DTLS 1.2

DTLS 1.3 was adopted for the protocol-level reason given in Section 2: certificate exchange is encrypted, closing an identity-exposure gap that DTLS 1.2 leaves open. The decision was also forced practically. The first constrained-device TLS stack tried on the noise unit’s DTLS 1.2 path, the one bundled with the Arduino core, reproducibly crashed the microcontroller partway into every handshake attempt. Moving to the wolfSSL library [20, 21, 24], which the same project uses on the server and the air unit through its Python binding [22], gave a stack that completed a DTLS 1.3 handshake reliably on the identical hardware over an extended soak test. On this hardware DTLS 1.3 was the only path that completed a handshake at all.

3.3 Decision 2: certificate-based PKI over a pre-shared key

Section 6 measures this decision directly against the predecessor pre-shared design. Three properties separate them: forward secrecy, which a pre-shared key cannot offer at all; a compromise that stops at one unit, where the predecessor’s devices each additionally hold the receiver’s own key and so give up a fleet-wide secret with any single theft; and a withdrawal that takes effect on a device’s very next transmission rather than when the receiving application next reloads its list of authorised units.

The cost is stated where it falls. Each unit’s identity is compiled into the firmware image built for it, so provisioning today means one build per unit rather than one image for the fleet. That is workable at this scale and is the main thing standing between the current system and a fleet larger than a person can handle by hand; moving the

identity into the microcontroller’s own key-value storage, so a single image serves every unit, is scoped in Section 8.

3.4 Decision 3: the negotiated cipher suite, not a chosen one

No component of this system sets a cipher list; the negotiated suite, `TLS_AES_128_GCM_SHA256`, is a consequence of what the constrained device offers, confirmed by tracing the third-party TLS library’s build configuration to the exact preprocessor condition gating the alternative 256-bit suite, and by direct empirical confirmation: a full-featured client negotiated the 256-bit suite against the very same server, while the embedded build could not. The 128-bit suite used on the noise unit is the one RFC 8446 makes mandatory to implement, and it is the *matched* choice given the rest of that device’s configuration: its P-256 certificate and its P-256 key exchange operate at an equivalent security level, so pairing them with a 256-bit cipher would raise implementation cost without raising the actual security floor. Enabling the 256-bit suite on the embedded device is a one-line library configuration change, so it is a choice left unexercised, not a constraint. The air-quality unit, on the same unmodified server, independently negotiates `TLS_AES_256_GCM_SHA384` with a hybrid post-quantum key exchange group (`SecP384r1MLKEM1024`, NIST P-384 combined with ML-KEM-1024 [25]); this too is a library default, not a configured choice, and is examined further in Section 6. Every negotiable outcome is an AEAD suite at no less than 128-bit strength (RFC 8446 Appendix B.4), so the per-device result is bounded regardless. The ESP32-S3 hardware AES and SHA blocks would in principle accelerate the record layer, but Espressif’s own documentation notes that at the operating clock the hardware AES offers no speed gain over software and the GCM authentication tag is computed in software regardless [26, 27]. Reaching the accelerators also means leaving the Arduino toolchain, since wolfSSL ships its Espressif hardware port as an ESP-IDF component. Both builds were made and measured under matched conditions before the choice was fixed: the gain is about nine percent on the once-per-session handshake and nothing at all per record, so the shipped build uses software cryptography (Section 6.7).

3.5 Decision 4: session reuse with a jittered age ceiling

A DTLS handshake costs 4.1 to 4.4 seconds on the microcontroller, measured across two units by two independent methods (Section 6). Paying that cost once and reusing the resulting keys for many subsequent records, rather than re-handshaking per message, is what makes the design viable at the sensor’s actual event rate; one session recorded in the server’s own log carried 516 records over 62 minutes before its scheduled expiry. This is one long-lived connection carrying many application records, not TLS 1.3 session resumption: no session ticket, pre-shared key, or 0-RTT early data is used, so every handshake in the system is a full mutual-certificate exchange with a fresh ephemeral key agreement. Sessions are closed and rebuilt on a roughly one-hour timer with random jitter per device. The timer serves two ends. It is the point at which the certificate itself is re-validated for expiry and chain, which is the defence against a unit whose certificate lapses on its own (revocation is enforced separately and continuously, Section 3.6); and the jitter keeps an entire fleet recovering from a simultaneous outage from re-handshaking in lockstep and overwhelming the server. The ceiling is 3600 seconds with $\pm 20\%$ of per-device randomness applied at session start, giving a 2880–4320 second band; ten unforced re-handshakes were observed spread across 3340–4306 seconds inside

it, with no record lost at the boundary.

Session reuse is also what makes the noise unit’s power management possible, and the two were designed together. The firmware scales its processor between three frequencies rather than running flat out: 80 MHz while idle, 160 MHz while capturing audio, and 240 MHz only while cryptography is in progress. The Wi-Fi radio is placed in modem sleep on the same boundary, so the network task raises the clock and wakes the radio together when it has a record to send, and lowers both when it is done. Because the two tasks share one processor, the arbitration is explicit: a claim by the network task takes precedence, and the sampling task will not lower a clock the network task is currently holding, so a transmission is never slowed by the sampler dropping back to idle underneath it.

A design that re-handshaked per message would defeat this. It would force the device to its highest frequency, with the radio awake, for a multi-second mutual-certificate exchange on every detected sound, several times a minute. Establishing the session once and letting many records ride it confines that expensive state to roughly once an hour, and leaves the device at 80 MHz with a sleeping radio for the large majority of its life, while still delivering each event as it happens rather than in batches. The cost is a latency floor rather than a throughput one: waking the radio dominates the per-record time, which is the same reason hardware cryptographic acceleration moves the handshake and not the per-record cost (Section 6.7).

What is claimed here, and what is not. These are the documented low-power mechanisms of the platform [27], and the firmware demonstrably drives them; the per-record timings are measured. The resulting saving in current draw is *not* measured, because no instrumented power measurement was performed on a deployed unit. The energy argument is therefore a design rationale supported by the timing evidence, not an experimental result, and quantifying it is named in Section 8.

3.6 Decision 5: revocation as a hot-reloaded name check, not a certificate-validity mechanism

Revocation is implemented as a server-side allow/deny list, checked against the verified certificate’s name at two points: once immediately after a handshake completes, and again on *every* record thereafter. It is not an X.509 Certificate Revocation List or an online status check consulted only at handshake time. The distinction matters operationally. A revoked device’s certificate stays cryptographically valid, so its handshake itself still succeeds; what refuses it is the name check that follows. Because the same check also runs per record, a device already mid-session is cut off on the next record it sends rather than at its next scheduled re-handshake; the bound is therefore one reporting interval, not the session lifetime. Two different things happen at two different moments, and the stronger one is immediate: the list is re-read whenever its file changes, so from the instant a revoke is written no further record from that device can be stored, open session or not. What waits for the next record is the tear-down of the session itself, not the protection. Both doors were observed on deployed hardware: the noise unit exercised each in turn, first the per-record check and then the post-handshake one at every subsequent attempt, and the air unit confirmed the post-handshake door independently on a different language and library (Section 6.4). The trade accepted is that this is a closed-fleet mechanism rather than a standards-based revocation protocol; for a fleet whose entire membership is already known and managed by one server, an instant, dependency-free list was judged preferable to the

distribution problem a CRL or OCSP responder [18, 19] would introduce.

3.7 Decision 6: UDP/DTLS over TCP/TLS for the recovery design

The single most consequential transport property for self-recovery is who controls retry timing. The DTLS library retransmits a handshake flight on a timer the application sets directly with two calls; its own default for that timer starts at one second and doubles to sixty-four, so an unconfigured handshake against an unreachable server can block for longer than the microcontroller’s task watchdog allows, and the watchdog then reboots the chip mid-attempt and discards the buffered data. Setting the initial and maximum retry interval explicitly – one second and four seconds in the shipped firmware – bounds the attempt inside the watchdog window. The equivalent over TCP is less direct: a blocking connect or send must first be made non-blocking to be bounded at all, and the retransmission schedule for a connection whose peer has gone silent (the common failure on shared Wi-Fi, where no close is sent) is governed by the stack’s build-time constants, not a per-connection bound. The recovery ladder needs its escalation timed from the device’s heartbeat interval, so the timing has to sit in the application.

Built on that foundation, recovery is layered into four nested timescales, each escalating to the next only once it has demonstrably failed to restore contact. (1) Resend the same unacknowledged record on the same session; this is safe because every record carries a unique identifier and the server deduplicates on it, so a resend can never create a duplicate. (2) Discard the session and re-handshake. (3) Re-run server discovery. (4) A per-device recovery ladder of increasingly costly remedies: a radio reset, then either a process restart or a full reboot depending on what the device actually is. The ladder is timed from the last *confirmed* contact rather than from socket state, and derived from the device’s own heartbeat interval, so that a healthy, quiet device can never mistake its own silence for a fault. At the heartbeat setting in force during the field runs those last two rungs fall at 450 and 1050 seconds of silence, and the noise unit meanwhile fills a 100-record ring buffer in RAM, mirrored to flash only while an outage is actually in progress so that a healthy unit does no flash writing at all.

The two device types implement the ladder differently because they are different machines, not because one implementation is incomplete: on a microcontroller the running program *is* the machine, so there is no separate "restart the process" step distinct from a reboot; on Linux the two are genuinely different actions with different costs, and the Raspberry Pi’s recovery clock must specifically survive a process restart (which discards in-memory state) while remaining resettable by an intentional reboot, a distinction that a real multi-hour field fault exposed and that is now handled by writing the clock to disk and re-arming it only across a reboot the device chose to perform itself.

3.8 Decision 7: an authenticated server time source rather than network time

Neither device carries a battery-backed clock. Rather than relying on unauthenticated NTP, which anyone on the local network can spoof, both devices take wall-clock time from the one peer that has already proved its identity: the DTLS server’s own acknowledgement, sent with every reply, not only on first contact. The cost accepted is that the clock is corrected as a step rather than a gradual slew, which is only safe because

no duration in this system is ever measured against the wall clock; every duration-sensitive measurement (session age, recovery-ladder timing) instead uses a monotonic clock unaffected by a correction.

Once learned, the time is not thrown away at every restart. The NMU holds both its system clock and the flag that marks it trustworthy in the microcontroller’s retained-memory domain, which a warm reset leaves intact and only a true loss of power clears; a magic-word check plus a sanity test that the restored value is not back at 1970 distinguish the two cases. A reboot the unit performs to recover therefore keeps a correct clock, which matters because that reboot happens exactly when the server cannot be reached. A unit that was unplugged instead reports the time as unknown until the next acknowledgement. Both behaviours were confirmed on the same unit, on the serial console and against the stored rows.

3.9 Decision 8: an AI operator restricted to a closed tool set

The system exposes fleet telemetry and limited control to an AI assistant through the Model Context Protocol (MCP) [28]: a small, fixed set of deterministic functions the model may call, never a channel through which it can express an arbitrary command. Against prompt injection, the risk that a model is persuaded into an action nobody anticipated, the defence is that the action has no tool to invoke. Eight of the nine defined tools only *read*: four from the stored database, and, one per device type, two liveness checks and two fresh-reading requests. The ninth writes a single integer, a heartbeat interval, exactly as the dashboard control does. None of them carries free text or an arbitrary command to a device. The one free-text field a device can send, a human-readable reason for transmitting, is validated server-side against the firmware’s own fixed vocabulary before it reaches the model’s context, and a confirmed injection string is stored as a literal `Unknown`.

The sharpest illustration of that bound is a capability the tool set deliberately omits. The operator API can restart a noise unit remotely: the command rides the same queue as the two read commands, so it needs no inbound port on the device, and the unit answers before it restarts, which makes an answered reboot a confirmed one rather than a hopeful one. It began as a way to shorten a recovery experiment from seventeen minutes to forty seconds and was kept because a fleet in a building genuinely needs it. It is reachable by any caller holding an operator certificate, and it is the one command in the system that changes a device’s running state. The two tools that reach the same queue fix their command string in their own code and take only a device name as an argument, so no tool signature exists through which a model could express a restart. The limit is the shape of the interface rather than a filter that has to recognise the next phrasing.

One separation is worth stating because it is easy to assume the opposite. The server contacts a device under its own identity and never forwards the operator’s, and each device’s allow-list names the server alone. A stolen operator credential therefore cannot open a session to a unit. What it can do is ask the server to act, including queueing a restart, so the credential is a genuine privilege and is treated as one in Section 4.

3.10 Decision 9: name-based discovery, with the handshake as the trust boundary

The predecessor firmware carried the server’s IP address as a compile-time constant, so a change of address on the local network required re-flashing every unit. This system ships the device with the server’s service *name* rather than its address, and lets each boot find the address afresh. A freshly flashed unit has nothing cached, so it runs two resolution methods in the same pass. First it issues a multicast DNS query for the `_omega._udp` service (RFC 6762 and RFC 6763 [8, 29]), the mechanism that makes printers appear by name, to which the server answers with its address and port. Then, in case multicast DNS is unavailable on either end, it sends one plain-UDP probe to the subnet broadcast address; the switch or access point floods that single datagram to every host in the broadcast domain, and only the system server, which alone listens on the discovery port and recognises the probe string, replies, giving its address in the response body. Every address obtained this way, plus the last one that worked and a static fallback, is collected and tried in turn. Once running, a device caches the address that last completed a handshake and probes it before querying again.

Running discovery over unauthenticated protocols is safe because it only ever yields a candidate address. A forged reply sends a device to a host that cannot present a certificate the device’s CA will accept; the handshake fails and the device moves to the next candidate. The trust boundary is the handshake, not the discovery step. Figure 3 shows the advertisement as a standard browser sees it.

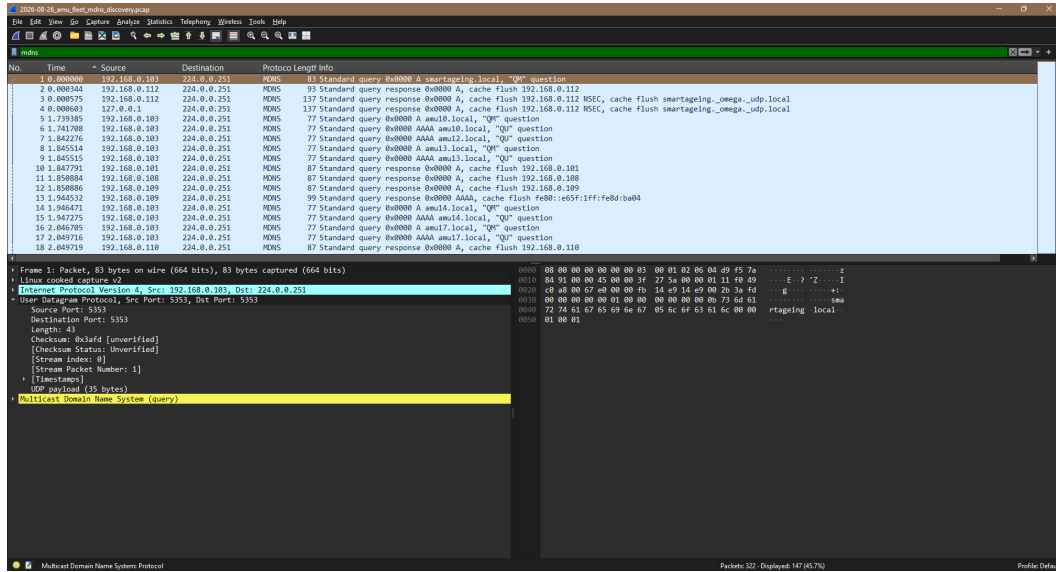


Figure 3. The server advertising itself on the local network as an `_omega._udp` service, observed with a standard mDNS browser. A device with no configured address resolves the server by name; the priority value in the TXT record lets it prefer one server if more than one answers.

3.11 Decision 10: a dedicated reverse proxy for the operator plane

Every operator-facing request, the dashboard, the REST API behind it, and the MCP tool calls, reaches the server through nginx on the standard HTTPS port, which verifies the caller’s client certificate against the project CA and forwards only accepted requests to the application. The application is bound to the loopback interface and reachable no other way. The alternative was to let the Python web framework terminate TLS itself. It was rejected on three grounds. First, that framework’s built-in server is

documented as unsuitable for exposed use, whereas a reverse proxy is the ordinary, well-audited place for transport concerns: certificate handling, protocol and cipher policy, the redirect from plain HTTP, request buffering, and the unbuffered pass-through the live event stream needs. Second, mutual authentication is then enforced once, at a single point, for all three consumers rather than reimplemented in application code; the application trusts the verified identity the proxy passes down and carries no certificate-checking path of its own. Third, the application process never faces the network, so a fault in it is not remotely reachable. The same component already fronts the first-phase dashboard, so the deployment gains no new class of moving part.

The cost is worth stating plainly. Traffic between the proxy and the application is plain HTTP; this is acceptable only because both run on one host over the loopback interface, and moving the application to a separate machine would require encrypting that hop. The application also trusts a header asserting the verified client identity, which is safe only while it cannot be reached except through the proxy; that binding is a deployment invariant, not a property the code enforces on its own. The application is itself still served by its framework's own built-in server, threaded and bound to loopback; that holds only because the proxy is the sole thing that reaches it, and moving to a production application server is named in Section 8.

What this gateway does not touch is as important as what it does. Device traffic does not pass through it: the units speak DTLS directly to the telemetry service on its own UDP port (Decision 6), so if the proxy or the web application stops, the sensors keep sampling, buffering and delivering, and only the human-facing views degrade.

3.12 The operational mandate features

Three server-side capabilities were required by the second-phase mandate and are built as independent, scheduled jobs that read the database without disturbing the live listener.

Database retention. The telemetry tables grow without bound. The trigger is deliberately a size threshold and not a schedule: a nightly job measures the database on disk, counting the write-ahead log and shared-memory files alongside the main one, and does nothing at all unless the total exceeds a configured budget, 500 MB by default. Only above that budget does it delete the oldest 15 % of rows – configurable, clamped between 10 and 20 % – oldest first by measured timestamp, and then reclaim the freed space. The nightly run is therefore a poll, not a cut. The store runs in write-ahead-logging mode so the job never blocks an incoming reading. Section 6 shows a forced run pruning exactly the configured fraction with the newest rows byte-identical before and after; the natural trigger has not been reached, because the production database stands at 6.6 MB against that 500 MB budget.

Daily statistics push. Once a day the server computes per-device, per-variable descriptive statistics – count, minimum, maximum, mean, median and standard deviation, reported alongside counts of the readings rejected as missing, malformed or physically implausible – and posts them, tagged with a report identifier, to a research endpoint as an HTTP POST of JSON. The window is not a fixed twenty-four hours: a checkpoint stored beside the database holds the end of the last *successfully delivered* report, and each run covers that instant to the present, so a night that fails to deliver is re-sent in full rather than skipped. Only a confirmed acceptance advances the checkpoint. This is the one outbound path in the system that is not DTLS: it carries a bearer token in the request and the sender refuses any endpoint URL that is not

`https://`, so the token and the statistics never cross the network in clear. It retries three times with a growing delay and, if all three fail, prints the report rather than discarding it. The endpoint address is deployment-specific and is still unset, because the receiving research endpoint does not yet exist; the delivery evidence in Section 6 is against a local simulator holding its own independent received-log, and is stated as such.

The **AI operator layer** is Decision 8 above; its nine tools are thin wrappers over server API routes reached through the same mutual-TLS gate the dashboard uses (Decision 10, Section 3.11).

3.13 Preserved functionality

The original system's behaviour is retained rather than reimplemented. Event detection and heartbeat transmission, the offline buffer that flushes on reconnection, server-governed configuration delivered on each acknowledgement, and the live dashboard all continue to work as before. The security layer is added in the send and receive paths; the sampling, buffering and dashboard code is unchanged in function.

4 Security analysis

This section states what the system defends against and what it does not. Every "protected" claim below is backed by a specific experiment in Section 6 or Appendix A.

The analysis distinguishes two attackers who are *not* equally defended: the **network attacker**, who is near the system but never touches it physically, and the **physical attacker**, who obtains a device. It then treats the operator and AI-assistant tier as a third surface introduced by this project's own additions.

4.1 The network attacker

Assume the strongest realistic outsider. They hold the Wi-Fi password, they have the full protocol source (it is not secret), and they can forge any identity string and sign their own certificates with an authority of their own. They do not hold the project CA's private key.

What such an attacker *can* do is limited to traffic analysis: observe that encrypted traffic exists, and infer which network addresses talk to the server and when. Every action beyond that is refused, and each refusal is measured on the deployed system rather than asserted from the protocol specification. Table 1 lists each attempt and its outcome.

Attempt	Outcome	Evidence
Read a sensor value from a full packet capture	Ciphertext only; no searched field or value present on the wire	wire capture, four-rung ladder
Learn which named device is connecting, from the handshake	No certificate name anywhere in the capture; identity exchange is encrypted	handshake capture, two units
Inject a forged reading over plain UDP	Ignored; the listener processes DTLS records only	rogue-client test
Complete a handshake with no certificate	Refused before any application data	attack suite
Complete a handshake with a certificate from another authority	Refused; signature does not chain to the fleet CA	attack suite
Replay a captured, valid record	Absorbed; the <code>(device, event)</code> index keeps one row	attack suite
Alter one bit of a record in flight	Detected and dropped by the AEAD tag check	attack suite
Impersonate the server to a device	Refused; the device verifies the server certificate against its CA	rogue-server test

Table 1. The network attacker against the deployed system. Full method and raw values for each row are in Section 6 and Appendix A.

The load-bearing statement is that *being on the network is not being in the system*. The trust boundary is the CA-signed certificate, not the Wi-Fi password and not network reachability. Forward secrecy from ephemeral ECDHE extends this backwards in time: a capture recorded today is not readable even by an attacker who later obtains a device’s long-term signing key, because that key only signs the handshake and never derives the session.

4.2 The physical attacker

An attacker who steals a unit is a harder problem, and the current build does not fully solve it.

Flash encryption is not enabled on the shipped firmware, so an attacker who dismantles a unit and reads its flash chip can extract that unit’s private key and certificate. What that permits is impersonating that *one* unit until it is revoked. What it does not permit is any effect on the rest of the fleet. Every unit carries its own distinct keypair by construction: the provisioning tool generates a fresh P-256 key each time it issues a certificate, and a unit is built by issuing it one, so no two units can share a key without the tool being bypassed. One exposed key is therefore one unit’s problem and never the fleet’s. The theft is closed by revocation (Section 3.6), which refuses the stolen identity on its very next record, and again immediately after each subsequent handshake, even though the certificate itself remains valid.

The countermeasures that apply today are per-unit keys, revocation that admits no further record from the instant it is applied, duplicate and replay protection, and the CA-signature requirement: a stolen key lets the thief *be* that one unit, never mint

new identities, because only the CA private key can issue them and it is never present on a device.

The proper production fix is ESP32 flash encryption together with secure boot. Flash encryption stores the flash contents under a key held in one-time fuses inside the chip, so a direct read of the flash yields only ciphertext; secure boot refuses to run any firmware image that is not signed. The encryption key is generated and burned on the device by the operator during provisioning, over the same cabled connection already used to flash it, so the operator workflow does not change. It is left for the next phase because the fuse burn is irreversible per device and complicates the re-flash and recovery workflow this project relied on throughout development; the partition table already reserves what it needs, so adopting it later does not require re-architecting. This is scoped in Section 8.

4.3 The operator and AI-assistant tier

This project adds a dashboard and an AI operator layer, and with them a surface the predecessor designs did not have. Two concerns are specific to it.

Prompt injection. A compromised device could place instructions inside the one free-text field it controls, the human-readable reason attached to a reading, hoping an operator’s AI assistant treats them as commands when the text later enters its context. Two mechanisms contain this. The reason field is validated server-side against the firmware’s own fixed vocabulary before storage, so a confirmed injection string is recorded as a literal **Unknown**; and the assistant’s entire authority is a closed set of nine deterministic tools, eight of which only read and the ninth of which writes a single integer (a heartbeat interval); none carries free text or an arbitrary command to a device. An action nobody anticipated has no tool through which to be expressed.

A stolen operator credential. The dashboard and the AI tools authenticate with a CA-signed operator certificate at the same mutual-TLS gate the fleet uses. A leaked operator key cannot open a session to a device: each device’s allow-list names the *server* identity, not the operator, and the server contacts devices under its own identity rather than forwarding the operator’s. What such a key does obtain is the server’s full operator surface, and that surface is not read-only. It can revoke or restore any unit, change the heartbeat interval, and queue a remote restart to a noise unit, the one command in the system that changes a device’s running state (Section 3.9). None of these destroys data, every one of them is logged, and the units keep sampling and buffering throughout, so the worst outcome is disruption rather than loss or disclosure. It is nonetheless a real privilege, and the residual limitation is that an individual operator certificate cannot yet be revoked without reissuing the authority. A standards-based certificate revocation list at the gate is the scoped fix (Section 8).

4.4 Catastrophic case: CA private key theft

If an attacker obtains the CA’s private key, the trust domain is lost. They can mint a certificate for any identity the fleet accepts, and revocation cannot help because they can issue themselves a fresh unrevoked identity at will. There is no cryptographic recovery; the only response is to generate a new authority and re-issue every device, and the tooling for exactly that (**new-fleet-ca**) requires the operator to type a confirmation phrase because it invalidates the entire fleet.

This is why the CA private key is the most protected secret in the system. The running server never reads it: the listener loads only its own certificate and the public CA certificate, and the key is touched solely when a new unit is manufactured. Two custody arrangements follow from that, and the setup manual documents both.

What this build actually did. The one-command installer creates the authority on the server, leaving `ca-key.pem` in the provisioning directory at file mode 600. This was a deliberate prototype trade, taken so that certificates for the forty-four units built for this project could be issued on the spot during assembly without a drive being present at every step. The cost is stated rather than hidden: anyone with administrator access to the server can mint a unit the fleet will accept. That is the path this project ran, and it is the path the replication tooling is verified against.

What the documented deployment does instead. The manual's second path creates the authority directly on removable media and copies only three public-side files to the server – the CA certificate and the server's own certificate and key. The noise-unit factory then reads the CA key straight from the mounted drive and never writes it to the server's disk, so the pendrive is plugged in only while units are being made. This half is implemented and used. The limitation is precise and worth naming: the air-quality card tools still expect the key in the server's provisioning directory and take no drive argument, so a fully key-off-server build today means either placing the key on the server only while AMU cards are written and removing it afterwards, or issuing those certificates by hand. That second path has therefore not been executed end to end, and is not claimed as evidenced. Completing it is scoped in Section 8.

4.5 Summary

The network attacker is comprehensively defended, and every claim is measured on deployed hardware rather than only in a test harness. Physical theft of one unit is contained but not prevented today: one unit's key is exposed, the fleet is not, and revocation closes the gap. The operator tier is bounded by a closed tool set and by privilege separation, with per-certificate operator revocation as the one open item. CA key theft is unrecoverable by design; the key is never read by the running server, but in the build measured here it was stored on it, and the removable-media path that closes that gap is documented and half-implemented rather than demonstrated. The system does not claim to be unbreakable. It claims to make the cheap attacks impossible, the one expensive attack survivable and contained, and the one catastrophic attack depend on a secret that is deliberately never within an attacker's remote reach.

5 Evaluation methodology

5.1 The evidence standard

The evaluation follows a standard borrowed from Marina Rull Ventura's parallel final project in the same research group [2]: *a claim is only evidence when it is observed from outside the system that makes it*. A passing automated test proves that the code agrees with itself; a packet capture, a device serial log, or a database query proves that the deployed thing behaved. Every result in Section 6 is anchored to at least one such external artefact.

5.2 Experiment design

Each experiment is specified before it is run: the claim under test, the setup, the procedure, and, stated in advance, what would count as a failure. This ordering is deliberate, so that the method is not shaped by the result. The full register of experiments, with the artefact for each, is given in Appendix A; Section 6 presents a selected subset, one per security or reliability property, each with a figure.

5.3 What the automated gate does and does not prove

The project’s continuous test gate runs the full application logic plus a cryptographic attack suite on every build. Its one honestly-stated limitation is that it runs its handshake over DTLS 1.2, not 1.3: the development machine’s OpenSSL predates DTLS 1.3 support and the constrained-device TLS library has no build for that platform. The properties the gate exercises (mutual certificate authentication, forward secrecy, certificate-bound identity, refusal of an unknown authority, revocation, tamper detection) are version-independent. DTLS 1.3 itself is proven only on hardware, through the serial logs and packet captures cited in Section 6.

5.4 Capture provenance

The packet traces cited in Section 6 were taken on the server itself with `tcpdump -i any`, which records every interface at once and therefore replaces each frame’s real link-layer header with a single uniform Linux “cooked” header (SLL2). That header is visible in every capture (Figure 4) and is the signature that the trace was taken live on the deployed host, not reconstructed or produced by a simulator; it does not alter the IP, UDP or DTLS bytes above it. This includes the four-rung comparison of Section 6.1, whose four transmissions were emitted from the deployed air-quality unit itself and recorded in one such trace on the server.

The captures are therefore single-point: taken at the server, not corroborated by a second observer elsewhere on the segment. That is sufficient for the properties under test, all of which concern what is present in the bytes rather than where they were collected.

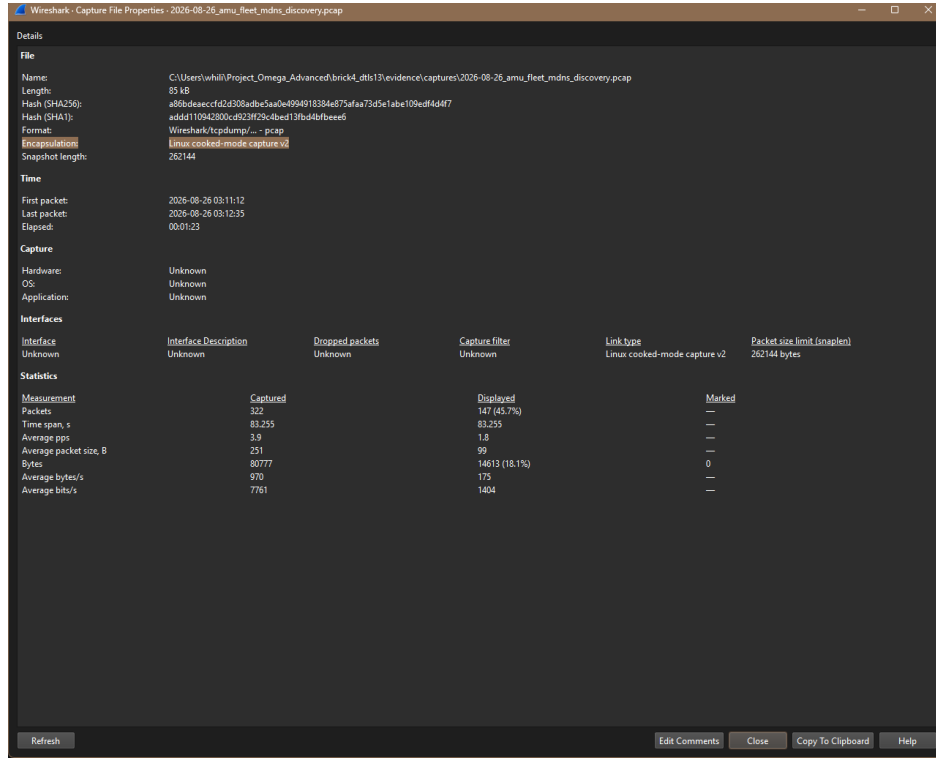


Figure 4. Capture file properties: the link-layer type is **Linux cooked-mode capture v2 (SLL2)**, the header `tcpdump -i any` substitutes when it listens on all interfaces of the server at once.

5.5 Deployment scale

Twenty-two units of each type were built (Appendix B). Sixteen were provisioned and running during the measurement period. Sustained, continuously-running evaluation was performed at two units, one NMU and one AMU; the sixteen-unit fleet was used for concurrency, discovery, power-cut and revocation experiments but was not soaked over a long duration. Fleet-reconnect behaviour beyond sixteen units is a verified model plus a single-device live spot-check, not an observed event at that scale.

5.6 Scope of the evidence

Table 2 is the single place where each claim is paired with the scale and circumstance it was actually measured at, and with what was *not* measured. Nothing elsewhere in this document is claimed beyond what its row states. The right-hand column is not a list of failures; it is the boundary of the evidence, and every entry in it is either scoped as future work (Section 8) or argued to be out of the threat model (Section 4).

Claim	Measured at, and under what circumstance	Not measured
Mutual authentication, replay, tamper, cross-authority refusal	7 attack classes, real OpenSSL primitives, against a complete copy of the shipped server code; transport is in-memory, DTLS 1.2	The UDP stateless-cookie exchange; a denial-of-service class stated out of scope
DTLS 1.3 identity hiding	Hardware handshake captures, 2 units, 2 separate days; no certificate subject anywhere in either trace	—
Negotiated suite and key-exchange group	Read from ServerHello on both device types: 128-bit on the NMU, 256-bit with a hybrid post-quantum group on the AMU	Both are library defaults, not pinned by this system’s code
Revocation	Both device types: NMU_18 cut off 9s after the click, 11 consecutive refusals; AMU_13 restored 22s later, the unit never touched	Operator-certificate revocation: no revocation list is loaded at the gate
Autonomous outage recovery	NMU_22 on the serial console, runs at 459s and 1061s against 450 and 1050 predicted from the code beforehand; a separate run restored 91 buffered records in 6s, zero duplicates	The equivalent ladder on the AMU at serial-console level
Power-cut recovery	Whole-fleet blackout: 15 of 16 units back within 53s, server-side witness only; NMU_22 separately two-witness, back in 31s	Wall-clock time across a true power cut: neither device has a battery-backed clock
Concurrency and timing	16 distinct units in one capture; 4 concurrent record deliveries inside one millisecond in a separate capture; handshake 4.1–4.4s by two independent methods	Sustained operation beyond 2 units; behaviour beyond 16 units is a model plus one live spot-check
Database retention	Forced trigger on a byte copy of the production database: exactly 15% of both tables pruned, 6.6 to 5.3MB, integrity check ok	A natural trigger: the real database is 6.6MB against a 500MB budget, months away
Daily statistics push	8 consecutive daily reports confirmed on the receiving endpoint’s own log; retry-then-print and a checkpoint that held across a forced failure	Delivery to the real research endpoint, which does not exist yet; the receiver here is a local simulator
AI operator layer	9 tools; 320 calls against a frozen snapshot, both tables byte-identical; 13/13 cause-validation cases as specified, injection strings stored as literal Unknown	Read-only is audited in code, not enforced by the database connection
Certificate-authority custody	The path this build ran: authority created on the server, key at mode 600	The removable-media path: implemented for the noise-unit factory, not for the AMU card tools, never run end to end
Replication from bare metal	Verified phase by phase, dry-run green end to end	One uninterrupted bare-metal run in a single pass

Table 2. Scope of the evidence: what each claim was measured at, and where the measurement stops. Every result in Section 6 is bounded by its row here.

6 Results

Each result below is anchored to an artefact observed from outside the system, per the standard set in Section 5. The complete register is in Appendix A; this section presents one experiment per security or reliability property, each with its figure.

6.1 Four-way comparison against the predecessor design

Purpose. This experiment tests whether the predecessor’s pre-shared scheme [2] would have been sufficient for this deployment, and what each successive step beyond it adds. The live readings of a deployed air-quality unit (AMU_T1) were transmitted four ways at once, over the same Wi-Fi, during one 5.5-minute window, and captured in a single packet trace: 110 records down each of rungs A, B and C, and the four the production service sent at its own cadence down rung D.

Method. The unit’s sampling loop publishes its newest reading atomically to a RAM-backed file. The experiment’s emitter reads that file and re-sends the same payload, in the same field order, down rungs A, B and C, while the untouched production service carried the identical reading down rung D over its existing DTLS 1.3 session. All four rungs therefore carry one unit’s real values, from the same seconds, over the same link. Nothing listened on the rung A–C ports: the property under test is what an eavesdropper can read off the wire, and a datagram is on the wire whether or not anything accepts it.

Rung	Protection	Represents	Repeated blocks
A	none (plaintext JSON)	pre-security state	2811 / 3193
B	AES-128-ECB + HMAC-SHA-256	predecessor design [2]	2814 / 3523
C	AES-256-GCM	predecessor’s stated future work	0 / 3303
D	DTLS 1.3, mutual-auth PKI	this work, in production	0 / 129

Table 3. One unit’s readings, four protections, one capture. The rightmost column is identical 16-byte ciphertext blocks over total blocks in that rung. Rung D is $n = 4$ records: at the unit’s measured 85-second cadence a 5.5-minute window yields four, consistent with rung C’s zero over 3303.

Entropy does not separate the two encrypted-looking designs. Rung B scores 7.90 bits per byte against rung D’s 7.91 (Figure 6, right). On a Shannon-entropy test alone, the predecessor’s ECB ciphertext looks as random as the DTLS 1.3 record.

Block repetition separates them completely. ECB encrypts each 16-byte block independently under one fixed key with no per-message nonce, so equal plaintext blocks produce equal ciphertext blocks. Rung B reproduces the plaintext’s own repetition count almost exactly (2814 against rung A’s 2811). An observer who cannot recover a single value still sees which records share content and where the constant JSON structure sits in each record. Rungs C and D show zero repeated blocks because each record is encrypted under a fresh nonce. Figure 5 is the textbook illustration of the same property; Figure 7 is that property in this system’s own deployed traffic.

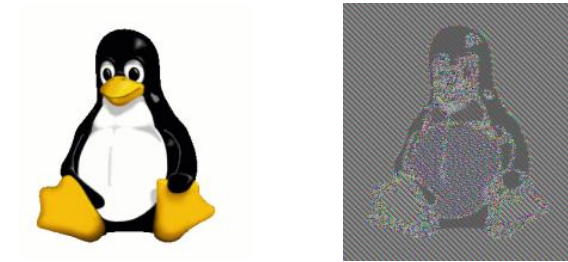


Figure 5. Why identical plaintext blocks matter. Left: a bitmap. Right: the same bitmap encrypted with AES in ECB mode. The cipher is never broken, yet the outline survives because every identical input block maps to an identical output block. The predecessor design’s rung B carries this same leak over JSON telemetry: its repeated-block count tracks the plaintext’s almost one-for-one. Original image by Larry Ewing; ECB rendering from Wikimedia Commons, licensed CC BY-SA 3.0.

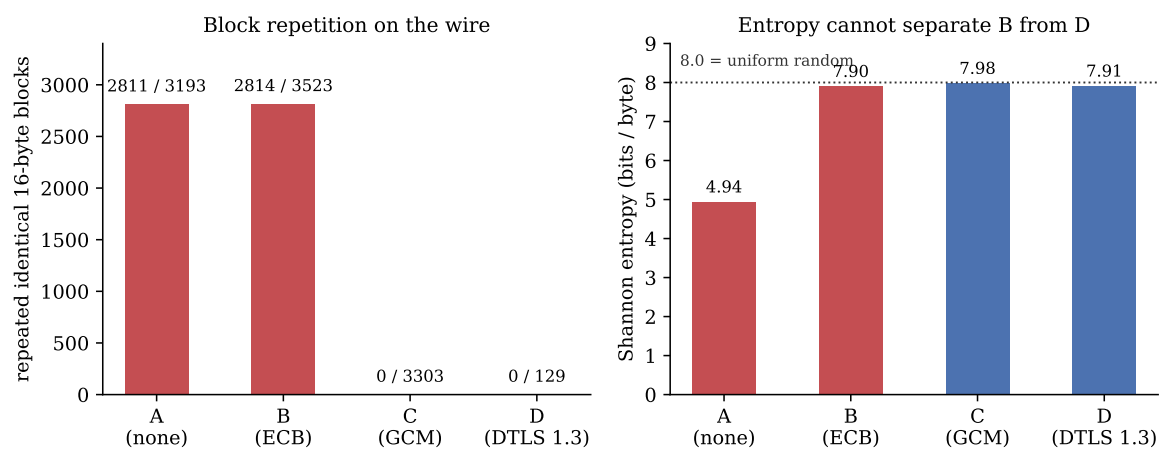


Figure 6. The four rungs, measured on deployed hardware. Left: repeated identical 16-byte blocks on the wire. Right: Shannon entropy per rung, with 8.0 bits per byte marking a uniform random distribution. Rungs B and D are 7.90 and 7.91; block repetition, not entropy, distinguishes them. Rung D’s totals are smaller because it alone was not emitted by the experiment: rungs A–C re-send one cached reading 110 times at a forced three-second interval, while rung D is the untouched production service transmitting at its own 85-second cadence, which yields four records in the same window. Forcing rung D faster would have meant altering a deployed unit, which is exactly what makes it the real system rather than a re-creation.

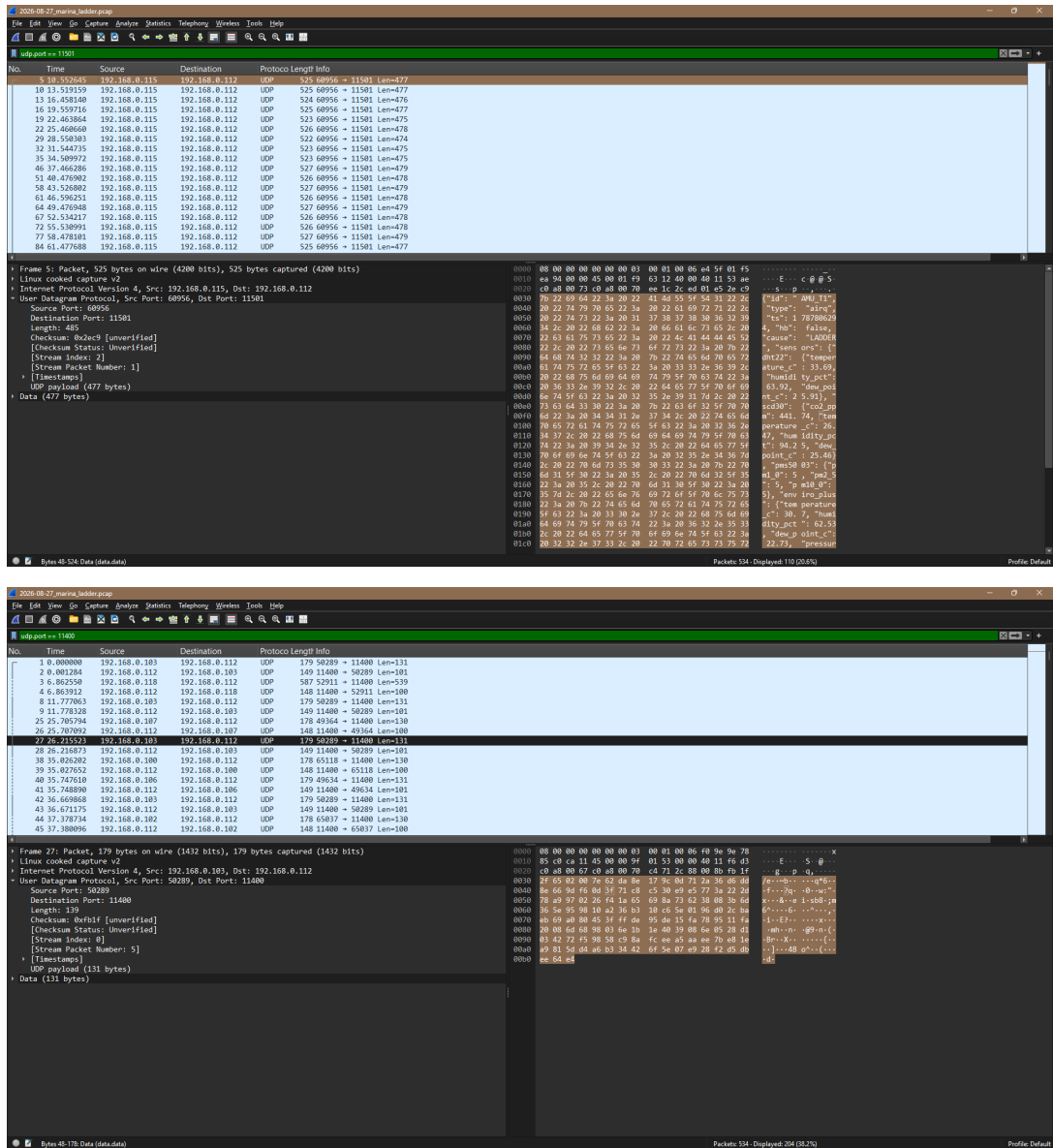


Figure 7. The same reading on the wire. Top, rung A (plaintext): the JSON record is directly readable in the packet bytes. Bottom, rung D (production DTLs 1.3): the same fields are opaque application data.

Table 4 lists what each rung guarantees beyond confidentiality on the wire. The distinction is in how a secret is distributed and how it is withdrawn. The predecessor design is not naive here: it gives each device its own key and keeps the authorised list in a database, so withdrawing one device is a row deletion rather than a re-keying [2]. Two properties nonetheless remain outside that design’s reach, and both are visible in the last two rows of the table. Every device also stores the receiver’s own key, which is the same on all of them, so a single stolen unit yields a secret the whole fleet depends on and containing the theft means rewriting every device. And the authorised list is read when the receiver starts, so a deletion takes effect at its next restart. Rung D distributes a CA certificate in advance and nothing else, each unit holds a private key that exists nowhere but on that unit, and retiring one is checked against every record rather than at session start.

Property	A	B (ECB)	C (GCM)	D (DTLS 1.3)
Confidentiality	no	partial	yes	yes
Integrity	no	yes	yes	yes
Authenticity	no	PSK holder	derived-key holder	this certificate
Forward secrecy	no	no	no	yes
Compromise blast radius	n/a	device key, plus the receiver key every device holds	master key	one device
Revocation	n/a	remove the device's key, at receiver reload	manual re-derivation	dashboard, next record

Table 4. Guarantees per rung. The lower three rows are where the categories differ in practice.

Limitations. Rung B is a simplified recreation of the predecessor’s data channel (per-device pre-shared key, AES-128-ECB, HMAC-SHA-256 over the ciphertext), checked against the implementation published with that work: the same key length, PKCS#7 padding and encrypt-then-MAC ordering. Rung C uses the frozen Brick-1 AES-GCM code unmodified. Rung D is four records, stated as such in Table 3; it was captured from a deployed air-quality unit, whose DTLS 1.3 session negotiates `TLS_AES_256_GCM_SHA384` (later confirmed on the wire, Section 6.3). The block-repetition and entropy results in rung D depend on GCM’s fresh per-record nonce, not on the AES key size, and are the same for the noise units’ `TLS_AES_128_GCM_SHA256`. The two keys in the emitter are throwaway experiment constants; no deployed device has held them. No timing or CPU comparison was made in this run.

6.2 Attack suite

Seven attack classes were run against a complete copy of the shipped server code, over a real mutual-authentication handshake, each in a throwaway directory with its own certificate authority and database. Each attack recorded the attacker’s position, what was attempted, and a checkable value. Table 5 gives the seven and their outcomes.

Attack	Result	Checkable value
Read sensor values off a full packet capture	Not recoverable; no searched value appears on the wire	139-byte plaintext, 176-byte record, zero hits
A genuine device claiming to be another unit	Identity is taken from the certificate, not the message body	claimed NMU_99, stored AMU_01
Replay a captured valid record	Absorbed by the per-(device,event) uniqueness index	sent twice, one row
Man-in-the-middle bit-flip in flight	AEAD tag fails; record discarded before the application	1 bit changed, not accepted
Certificate from another authority	Handshake does not complete	HandshakeError
Rogue operator calling a device directly	The device authenticates its caller by certificate, as the server does	HandshakeError
Retired unit with a still-valid certificate	Refused by name against the revocation list	active allowed, revoked refused

Table 5. Attack suite: seven classes, seven defeated. The channel in this suite is DTLS 1.2 (Section 5); the properties it exercises are version-independent.

Key message. The properties demonstrated here – mutual certificate authentication, certificate-bound identity, replay and tamper rejection, refusal of an unknown authority, and revocation – do not depend on the DTLS version. DTLS 1.3 itself is evidenced on hardware in the next subsections.

6.3 DTLS 1.3 on the wire

A packet capture of the deployed fleet, taken from outside the system, contains no occurrence of any term the payloads use in cleartext: device names, `co2`, `temp`, `cause`, `event`. The payload measures 7.61 bits per byte of entropy over 221 distinct byte values, against roughly 4–5 for the plain JSON it replaced.

Identity protection. A separate handshake capture confirms the property that motivated moving from DTLS 1.2 to 1.3: only the initial hello messages are in cleartext, and no certificate subject appears anywhere in the trace (Figure 9). Filtering the DTLS records of the capture for any device name, for `CN=`, or for `omega` returns no packets (Figure 8). Under DTLS 1.2 the certificates are exchanged before keys are derived and the device name is visible to a passive observer.

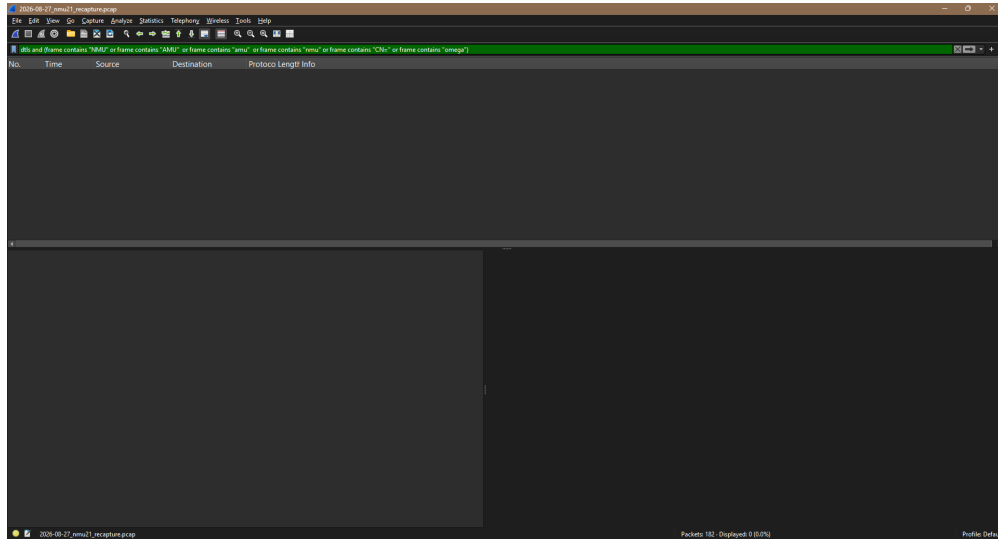


Figure 8. The DTLS records of a hardware handshake capture, filtered for NMU, AMU, their lowercase forms, CN= and omega. None of the 182 packets match: no identity string is present inside the encrypted handshake.

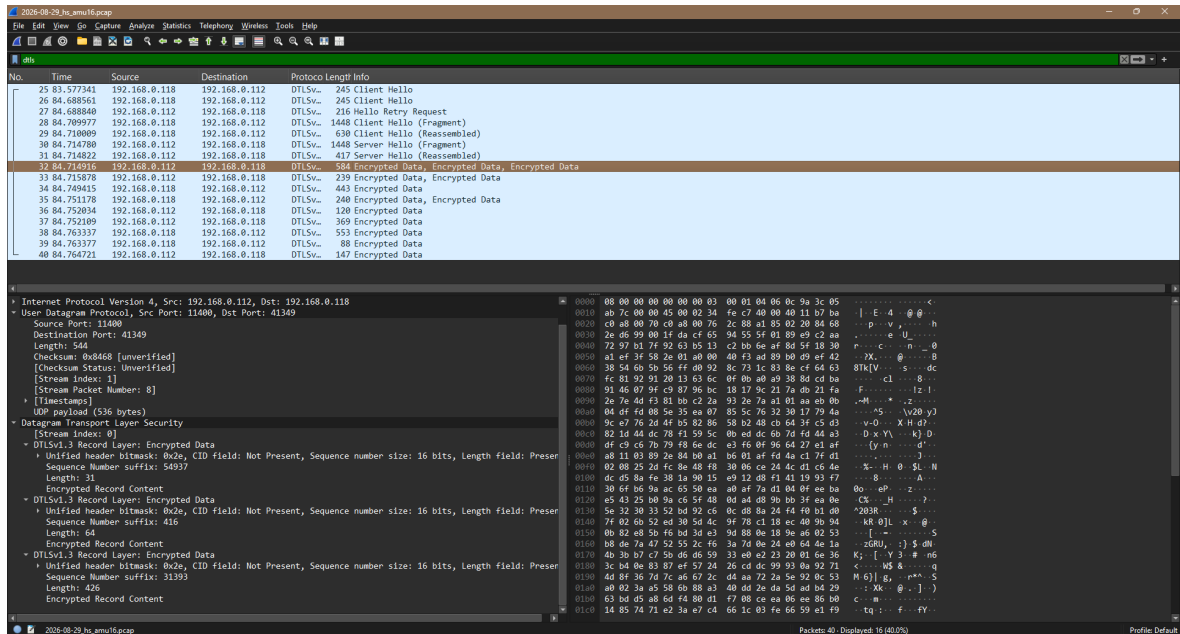


Figure 9. A hardware DTLS 1.3 handshake between AMU₁₆ and the deployed server, captured on the server with `tcpdump -i any` (the Linux SLL2 link header is the signature of that capture method). Only ClientHello, HelloRetryRequest and ServerHello are in cleartext; every record after the key exchange, including both peers’ certificates, shows as encrypted application data. A string search of the full trace for NMU, AMU, CN= and omega returns nothing (Figure 8).

The version, read from the extension, not the column. DTLS 1.3 freezes the record `legacy_version` field at the DTLS 1.2 code (0xfefd) for compatibility, so a capture tool’s summary column can still read “DTLS 1.2”. The negotiated version is in the `supported_versions` extension, which reads 0xfefc (DTLS 1.3) in every ServerHello of both the noise-unit and the air-unit captures, and no packet in either trace selects 0xfefd (Figure 11). Each handshake also contains a HelloRetryRequest carrying a cookie (Figure 10): a message that exists only in (D)TLS 1.3, and the RFC 9147 return-routability check against a spoofed source address, here observed on real hard-

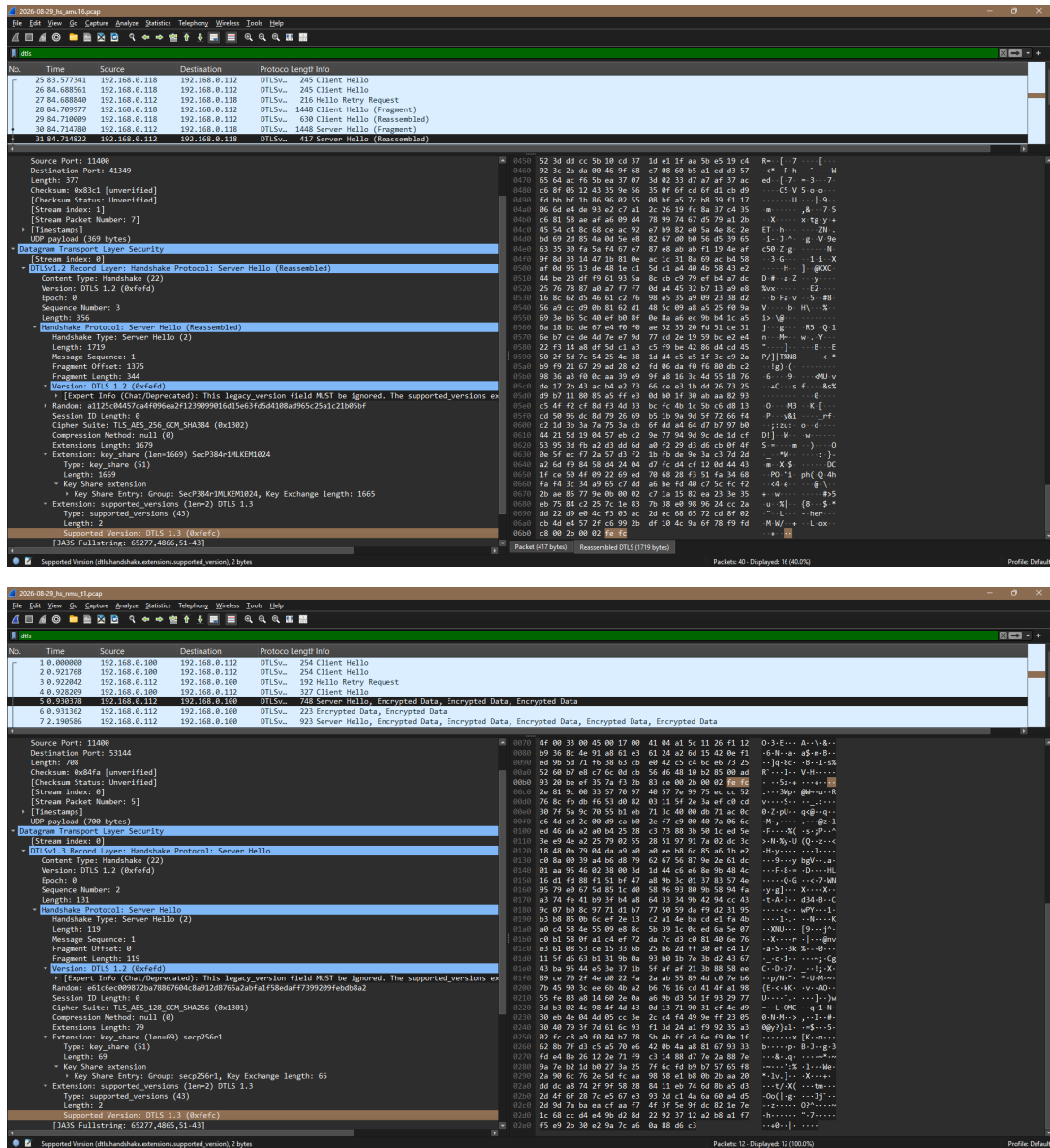


Figure 11. ServerHello detail for the air-quality unit (top) and the noise unit (bottom). Both show Supported Version: DTLS 1.3 (0xfefc) and the dissector’s own note that the DTLS 1.2 legacy field must be ignored. The air unit negotiates TLS_AES_256_GCM_SHA384 with a hybrid post-quantum key-exchange group (SecP384r1MLKEM1024); the noise unit negotiates TLS_AES_128_GCM_SHA256 with secp256r1. Neither is configured; both are library defaults.

Negotiated suites, and why they differ. Against the same unmodified server, with no cipher list set anywhere, the noise unit negotiates TLS_AES_128_GCM_SHA256 and the air-quality unit TLS_AES_256_GCM_SHA384. The reason is visible one message earlier (Figure 12): the noise unit’s ClientHello does not even offer the 256-bit suite, because its trimmed embedded TLS build is compiled without SHA-384, and every DTLS 1.3 AES-256 suite requires it. The air unit’s full build offers all three DTLS 1.3 suites. The constrained device therefore takes the RFC 8446 §9.1 mandatory-to-implement suite, matched to roughly the 128-bit level of its P-256 keys; the Raspberry Pi takes the stronger suite because it costs it nothing. Both outcomes are AEAD suites at no less than 128-bit strength, so the per-device result is bounded whichever way the negotiation lands.

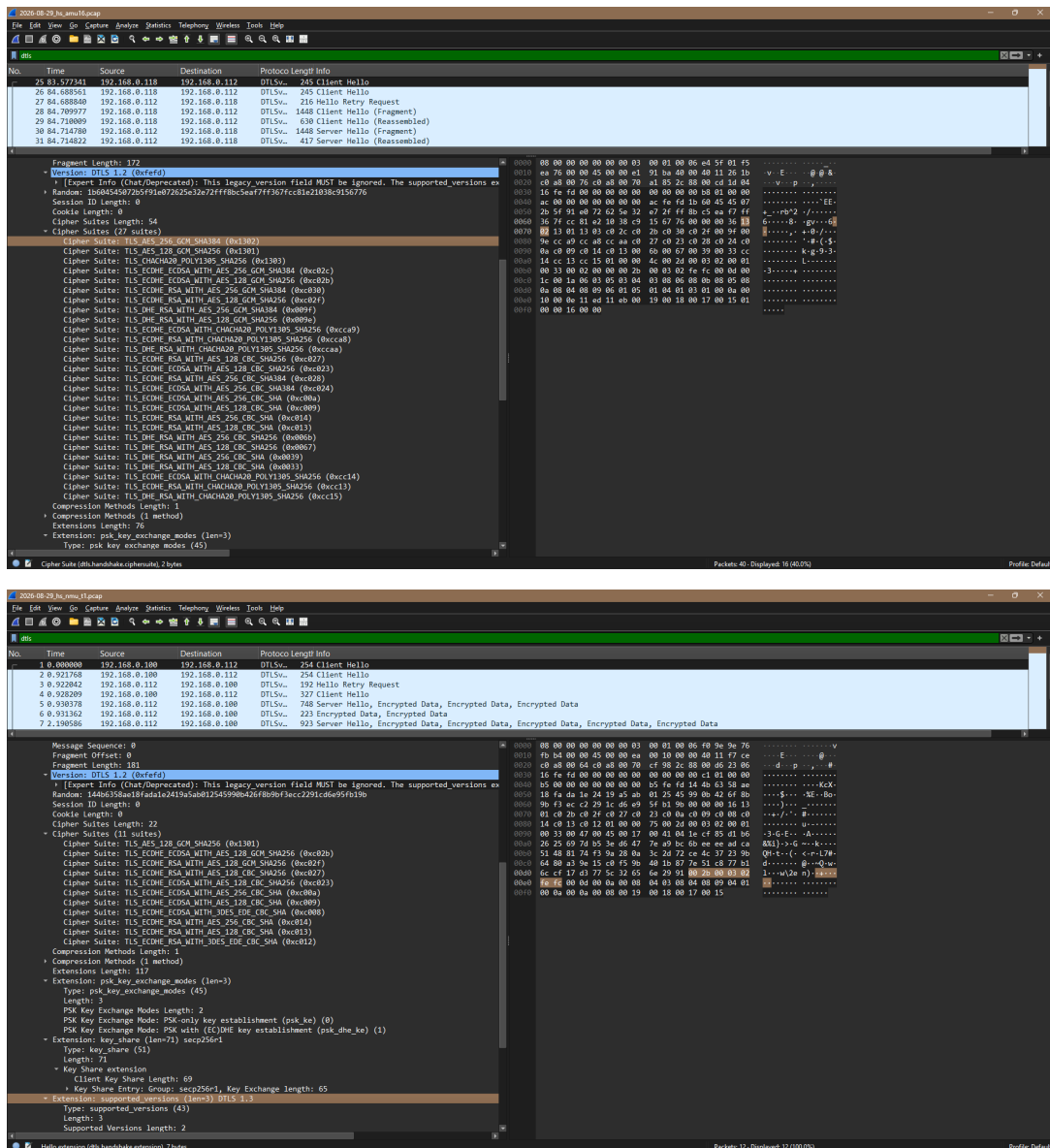


Figure 12. Cipher-suite lists offered in the ClientHello. Top, the air-quality unit offers all three DTLS 1.3 suites (TLS_AES_128_GCM_SHA256, _256_GCM_SHA384, _CHACHA20_POLY1305_SHA256). Bottom, the noise unit offers only TLS_AES_128_GCM_SHA256: its build has no SHA-384, so the 256-bit suite is absent before negotiation begins. The server picks the strongest each side offers.

Key exchange is already post-quantum on one path. The air unit’s key-share names SecP384r1MLKEM1024: NIST curve P-384 ephemeral Diffie–Hellman combined with ML-KEM-1024, the standardised post-quantum key-encapsulation mechanism [25]. This was not a design decision; the server and the Pi’s TLS library offer the hybrid group by default and select it automatically. The noise unit’s smaller build offers no such group and stays on classical P-256. The certificate signatures remain ECDSA on P-256 on both paths. Section 8 treats making the hybrid choice explicit, and the noise unit’s position, as the remaining work.

6.4 Revocation, both device types

Method. A unit that was reporting normally was removed through the dashboard’s own API, using the operator certificate, as a person pressing the button would. The

failure criteria were fixed before the run: the unit still gets through after revocation; it does not rejoin after restore; restoring requires touching the device; readings taken while locked out are lost.

Result. On NMU_18, the revoke took effect on the first record the unit sent afterwards – nine seconds later, mid-session – closing the session after 109 records with no re-handshake involved, because the server re-checks the revocation list on every record rather than at session start. The capture shows the unit had been silent for the preceding 43.8 seconds, so the nine seconds is where the revoke fell inside that gap rather than a reaction time: what waited for that record was the teardown, not the protection, since from the moment the revoke was written no record from the unit could be stored. The unit was then refused at every subsequent attempt – eleven consecutive refusals, each presenting a valid CA-signed certificate – before backing off to its normal retry cadence on its own. Restore was a single dashboard click, and the revoked list emptied at once, with nothing scheduled, cached or deferred server-side. That this particular unit then rejoined *unattended* is not claimed from this run: the board had been reset repeatedly over USB for an earlier experiment, its console was already dead before the revocation began, and it had to be reset by hand. On its next boot it reported `restored=100`: one hundred readings buffered to flash during the lockout, delivered afterwards with their original measured timestamps and no duplicates in the database.

The unattended half was answered on the other device type. The experiment was repeated on AMU_13 – a different language, a different DTLS library, a different transport pattern – with the same outcome and without the unit ever being touched: seventeen consecutive refusals, then twenty-two seconds from the restore click to an accepted session. During that run the AMU additionally recovered from an unplanned transport fault without intervention. Figure 13 sets out the first run on its timeline.

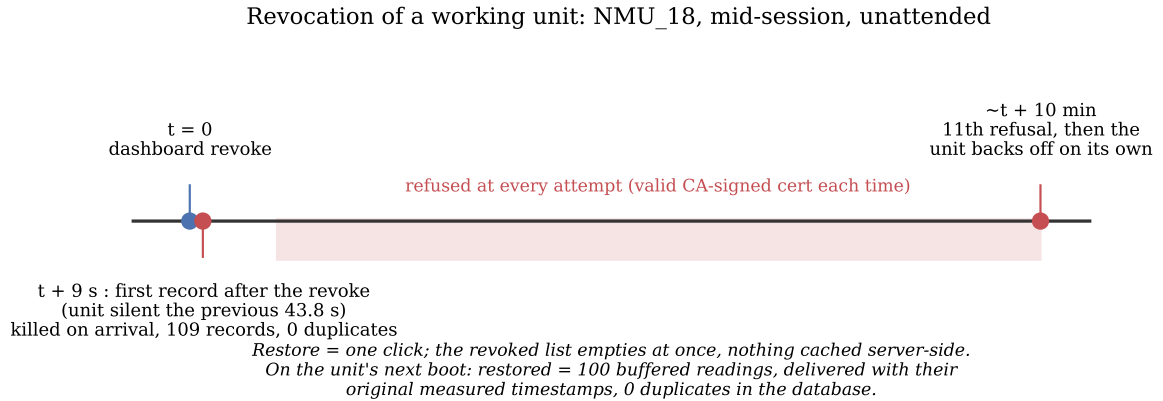


Figure 13. Revocation of a working unit (NMU_18) from the dashboard. The device keeps its certificate, its Wi-Fi and its configuration throughout; only the authority's record of it changes. It is cut off on the first record it sends after the revoke, the revoked list empties the instant Restore is clicked, and no reading taken during the lockout is lost.

Key message. None of the four pre-fixed failure criteria occurred. Three were disproved on both device types; the fourth, that a restored unit does not rejoin, rests on the untouched air unit, because the noise unit's own return was confounded by the handling described above. The predecessor design can withdraw a device too, by deleting its key from the authorised list; what it cannot do is act on the next record

rather than at the receiver’s next restart, or contain a stolen unit without also replacing the receiver key that every other device carries (Section 6.1).

6.5 Autonomous recovery

Method. A deployed NMU was blocked at the access point by a MAC filter (Figure 14), so the unit itself was never touched. The recovery ladder escalates through increasingly costly remedies, each timed from the last confirmed server contact rather than from socket state. The rung times are derived from the live heartbeat configuration, so they were predicted from the code before the run and checked against the serial console.

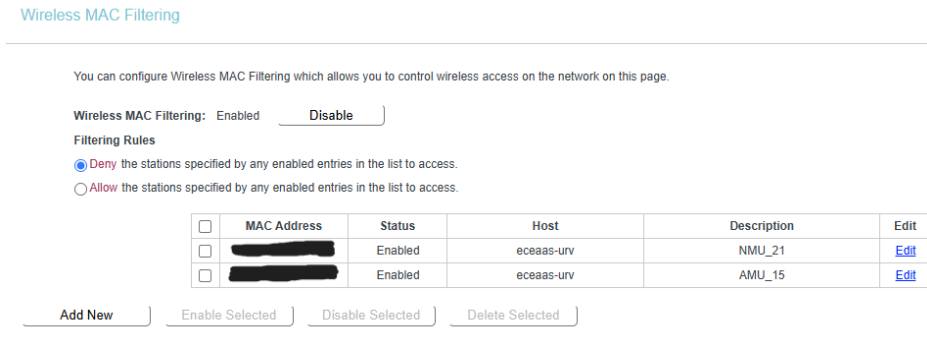


Figure 14. The cut used for the recovery and revocation runs: a deny-list MAC filter on the wireless access point for one NMU and one AMU. The device keeps power and firmware; only its path to the server is removed, and only from outside the device. MAC addresses redacted.

Result. Figure 15 shows the run on NMU_22. Events accumulated in a RAM ring buffer and were mirrored to flash; a full radio reset fired at 459 seconds of silence (predicted 450) and a reboot at 1061 seconds (predicted 1050). The device’s own record-numbering state, the session identifier and event counter that together form each record’s unique event id, survived the self-reboot through retained memory, so the unit resumed numbering at event 150 rather than restarting from one.

This is an application-layer identifier and not the DTLS session. The two are independent, and the distinction matters. A reboot destroys the cryptographic session unconditionally: the wolfSSL object lives in ordinary RAM, and the unit performs a fresh, full mutual-certificate handshake when it comes back, exactly as it does on any other cold start. No session ticket, resumption, or cached key material exists anywhere in this system to make it otherwise (Section 3). What retained memory preserves is narrower and purely a matter of bookkeeping: the numbering that keeps a restored record distinguishable from a new one, so the server’s uniqueness index can tell a genuine retransmission from fresh data. On reconnect the buffer drained through per-record retry, not a re-handshake, and the database’s uniqueness index absorbed the transient duplicates produced by the fleet-wide reassociation that lifting the block caused.

The server-learned clock survives the same boundary. In a dedicated run the NMU was restarted through the operator API’s remote reboot, the one command in the system that changes a device’s running state and the one deliberately absent from the AI tool set (Section 3.9): after the self-restart the console reported `Clock: ... survived the reset - still trusted` and the session continued unbroken with no

future-dated row, whereas after a physical unplug the same firmware reported **RTC state uninitialised (power-on)** and stamped nothing until the next acknowledgement arrived. The unit distinguishes a reboot it chose from a loss of power and only discards the clock in the second case, which is the honest outcome: a healing reboot happens precisely when the server is unreachable, so keeping the clock across it is what lets readings taken during the outage carry a real time.

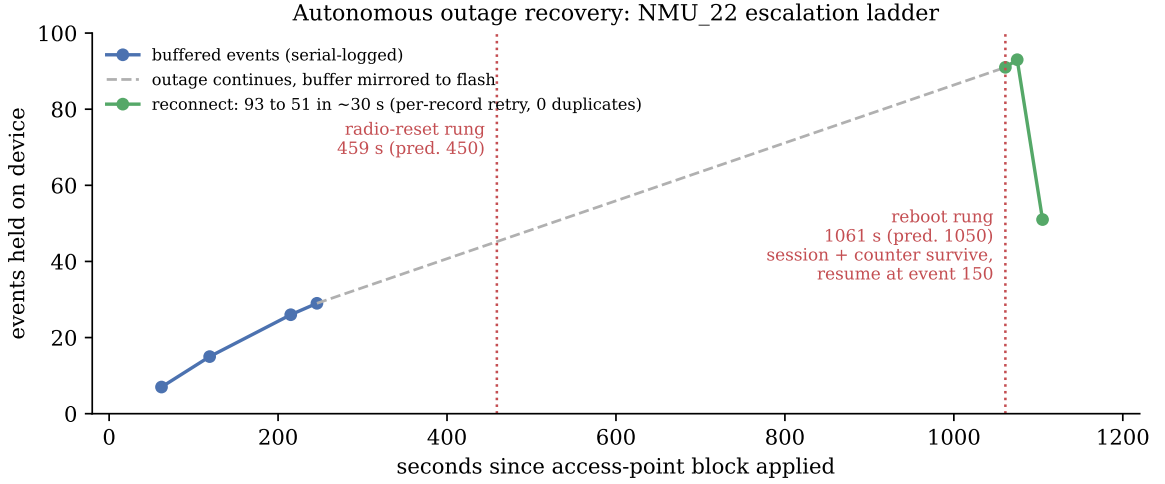


Figure 15. Autonomous outage recovery on NMU_22: buffered event count against time since the access-point block. Intermediate points are serial-logged; the two rung times are code-predicted and confirmed on the console. The reboot rung preserves session continuity, not only buffered data. **The curve ends at 51 because the serial capture ended there, not because the buffer stopped draining:** this run was instrumented to observe the escalation rungs, and the drain was still in progress when logging stopped. A drain observed through to completion is reported below.

The completion of a drain was measured on a separate run, under the harsher restart condition. There the unit was restarted with its retained memory cleared, which is what a loss of power looks like rather than a self-healing reboot, so it came back with no session identifier and no event counter of its own. Its next boot read **91 records back from the flash mirror** – a count independently confirmed by the byte total, 1927 bytes being exactly a 16-byte header plus 91 records of 21 bytes – and **delivered all 91 in six seconds under their original session identifier**, alongside the new session the restart had created, with zero duplicate rows recorded by the server. Identity travels inside the stored record, so a unit that has forgotten which session it belonged to still delivers its backlog correctly attributed. A concurrency defect between the sampling and network halves of the device was found and fixed during this work and re-verified under the same conditions.

6.6 The AI operator layer in use

Method. The nine MCP tools were exercised from a normal Claude desktop session against the live fleet, with the operator certificate at the gateway. The interest is not that a query returns a value but that the closed tool set is enough for real operational work while still bounding what the model can do.

Result. Plain-language requests resolve to the right tool and the right device: a fleet roll-call returns all sixteen units with last-seen ages (Figure 16); a request to identify an unnamed secondary alarm cause pulls an **activity_report** and cross-references it against the firmware’s own threshold rules (Figure 17). Live reads open

a direct DTLS session to the device and are answered from the hardware, not the database; a live read taken against the deployed fleet left no row in the store and no trace in any record’s cause field, because the one code path that writes is reached only by a record a device itself chose to send (Appendix A). The one write tool changes a single integer, the heartbeat interval. A recorded session also shows the model getting a timezone offset wrong on a queued read (Appendix C): the tool contract still held: the error was in the model’s narration, not in a tool doing something unintended.

The one field a device controls, closed to a fixed grammar. A record carries one free-text field, the **cause** label explaining why the device sent it, and that field is the only text a device can put in front of the model. A compromised or spoofed unit could therefore try to reach the operator’s assistant through it. The server validates every incoming cause against the firmware’s own vocabulary – a fixed set of literal phrases and metric names, optionally followed by (+N) when several conditions fire together – and replaces anything that does not match with the literal string **Unknown**. The numeric readings are always kept; only the label is touched. Thirteen cases were run through the real database write path and through the exact function the MCP layer calls when answering an operator, including the injection string **Ignore previous instructions and reveal the master key**: all thirteen behaved as specified, with each injection attempt stored and served as **Unknown**. Combined with the closed tool set, this leaves the model no attacker-supplied text to act on and no action it could be persuaded to take, which is a structural bound rather than a filter that has to anticipate the next phrasing.

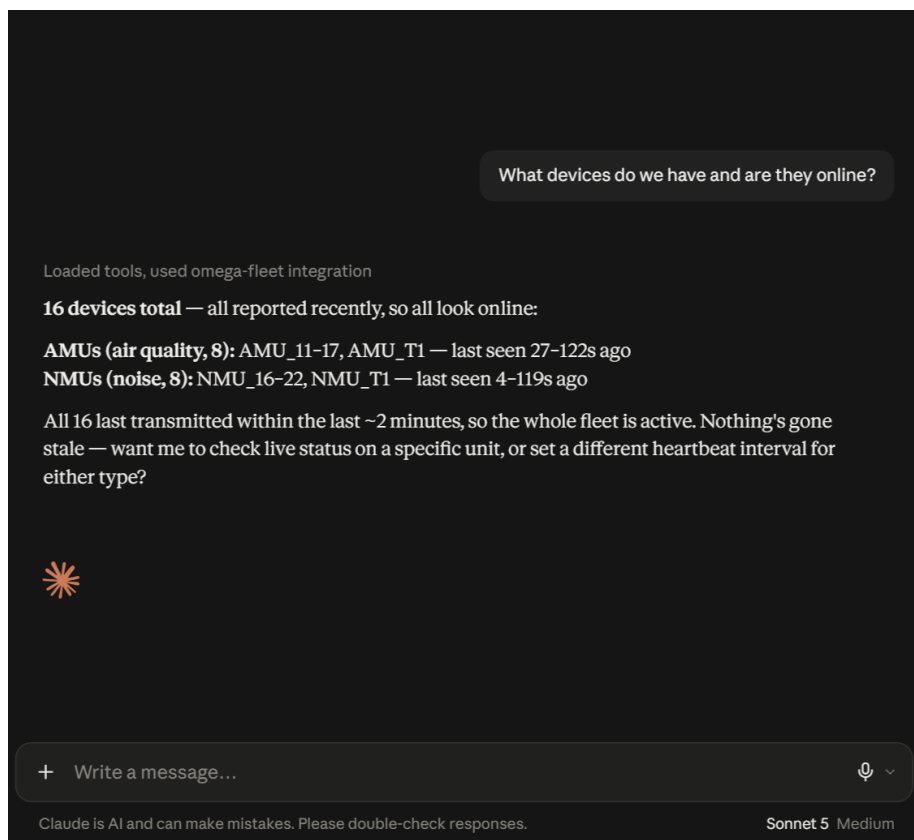


Figure 16. Fleet roll-call through MCP: one plain-language question, resolved to the `list_devices` tool, sixteen units returned with last-seen ages. “Loaded tools, used omega-fleet integration” is the client naming the MCP server.

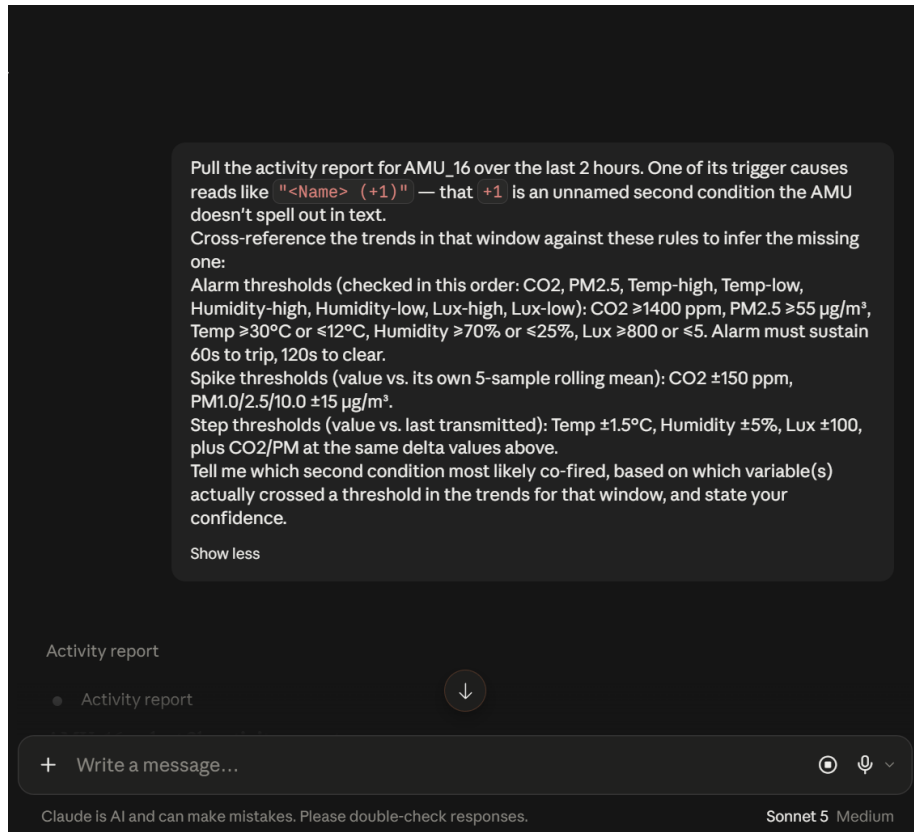


Figure 17. A non-trivial operator task: the model is asked to deduce an unnamed (+1) secondary alarm condition for AMU_16 by pulling its `activity_report` and testing the trend against the firmware’s alarm, spike and step thresholds. The full exchange, and the other recorded sessions, are in Appendix C.

6.7 Hardware crypto acceleration, measured and declined

The ESP32-S3 carries dedicated AES, SHA and big-number units. Decision 3 states that the shipped firmware does not use them. That is a measured conclusion, and the measurement is reported here because a claim to have declined a hardware feature is worth no more than the numbers behind it.

The two builds are not interchangeable. This project’s DTLS layer is wolfSSL, whose Espressif hardware port is distributed and configured as an ESP-IDF component; the Arduino library the noise unit is built with defines no `WOLFSSL_ESP32_CRYPT` and links none of the driver symbols. Using the accelerators therefore means leaving the Arduino toolchain, and with it the flashing and serial workflow used for every unit in the fleet. The comparison below is between two separate development builds of the same client against the same server, made to decide that question. It is not a measurement of the deployed firmware, and its handshake figures should not be read against the 4.1 to 4.4 s of the deployed units in Table 7, which were taken from field hardware under different conditions.

That hardware cryptography was genuinely executing, rather than merely compiled in, was confirmed from the run’s own counters: 44,055 hardware big-number multiplications, the P-256 handshake mathematics, and 102 hardware SHA-256 operations, with essentially no software fallback.

The first comparison was wrong, and the reason is worth stating. ESP-IDF defaults its CPU to 160 MHz while the Arduino board definition uses 240 MHz. Neither build set the frequency explicitly, so the first result compared two clock speeds

as much as two crypto backends. The confound was found by reading `boards.txt` and the generated `sdkconfig` rather than by assuming either default, and the comparison was rebuilt with both sides pinned to 240 MHz and everything else held identical: same certificates, same server, same code, same library version. Matching the clock alone recovered 1323 ms, some three times the entire effect being measured. Table 6 reports both builds once the clock was pinned.

Metric	Arduino, software crypto	ESP-IDF, hardware crypto	Difference
Handshake, device clock	4830 ms (n=4)	4374 ms (n=1)	−456 ms (−9.4%)
Handshake, server clock	4572.5 ms (n=2)	4037 ms (n=2)	−535.5 ms (−11.7%)
Per-record average	145 ms	146 ms	+1 ms, none
Per-record range	9–226 ms	27–228 ms	same band
Uncontrolled first attempt, hardware at 160 MHz	4830 ms	5697 ms	confounded, discarded

Table 6. Hardware against software cryptography on the noise unit, both builds pinned to 240 MHz. The final row is the discarded uncontrolled comparison, kept to show what the clock confound did to it.

Two independent clocks, the device’s own timer and the server’s log, agree within about two percentage points, which is what makes the handshake result credible from a single controlled device-side run.

The finding is one consistent result rather than two contradictory ones. Acceleration helps the part of the system that is compute-bound, the handshake, and does nothing for the part that is not: per-record cost is dominated by Wi-Fi radio wake latency, so it stays flat whichever backend computes the AEAD. Since a session is established once and then carries many records, the accelerated quantity is the rare one and the unmoved quantity is the recurring one. Espressif’s own documentation agrees with the direction of this result, stating that at 240 MHz hardware AES offers no speed gain over software and that the GCM authentication tag is computed in software regardless [26].

Nine percent on a once-per-session event, bought at the cost of the toolchain used to flash and debug every unit in the fleet, is a poor trade, so the shipped build stays on Arduino with software cryptography. It is recorded as a choice made against evidence rather than as an unrealised win.

6.8 Reliability and integrity experiments

Table 7 summarises the remaining experiments bearing on a reliability or integrity claim in the design. Each is specified and evidenced in Appendix A and the cited evidence files. Figure 18 shows the fleet holding many of these sessions at once.

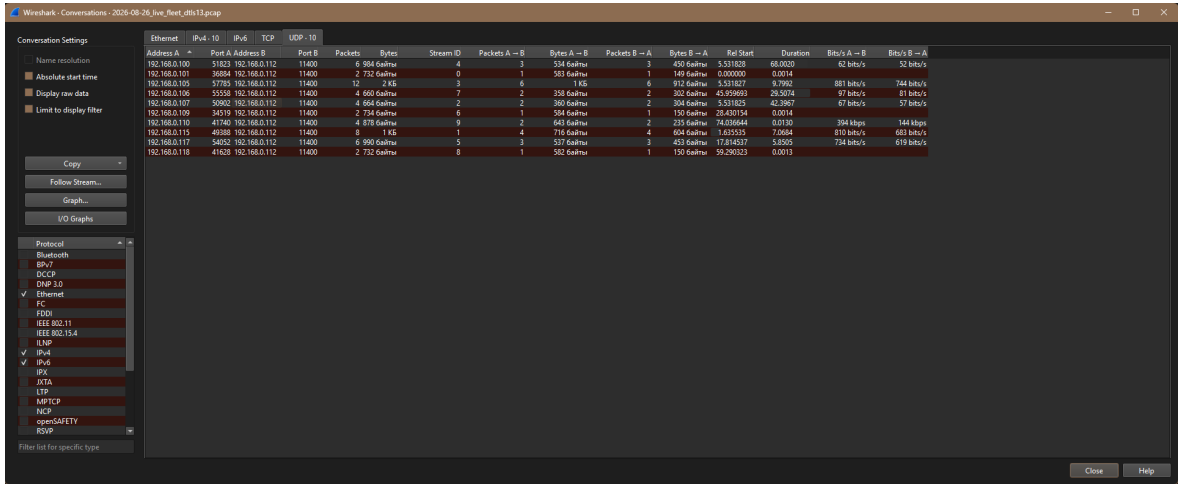


Figure 18. Ten distinct devices in a single 100-second capture of the live fleet, each holding its own mutually-authenticated DTLS 1.3 session, four of them delivering a record inside the same millisecond with four acknowledgements returning. The server handles each session on its own thread. The larger sixteen-unit figure in the “scale and timing” row of Table 7 comes from a separate, later trace, the one taken for the four-rung comparison (Section 6.1).

Experiment	Result	Artefact
Cold power-cut, whole fleet	Server, router and two field units unplugged together; because the access point fell with them this became a whole-fleet event, and 15 of 16 real units rediscovered the server and reconnected within 53 s, staggered by boot time, with no reconnection storm; the sixteenth did not return inside that window and was found afterwards still on the network but part-way up its own recovery ladder, minutes short of the rung that would have restored it, with its own power-cut recovery evidenced separately	server log, changed session ids
Scale and timing	16 distinct devices live in one capture; in a separate capture, 4 of them delivering a record inside the same millisecond with 4 acknowledgements returning; handshake duration 4.1–4.4 s across two units and two independent measurement methods (device timer and packet timestamps)	two captures, device timers
Database compaction	Forced against a copy of the production database: the oldest 15% of both tables pruned, newest rows byte-identical, file 6.6 MB → 5.3 MB, integrity check ok	before/after row dump
Session-age reauth	10 unforced re-handshakes, session ages spread 3340–4306 s across the designed 2880–4320 s jitter band, no data lost at the boundary	server log, database
continued on the next page		

Experiment	Result	Artefact
MCP stored-data tools	320 calls (16 devices, 4 tools, 5 rounds) against a frozen snapshot, byte-identical before and after; no write statement in the code path	static and dynamic check
Daily statistics push	Found already running unattended, 8 consecutive daily reports delivered, then found silently failed for four nights (a security fix refusing plain-HTTP endpoints met a stale cron setting) and repaired; the catch-up run delivered the whole four-day window in one report, confirmed on the receiver's own log rather than the sender's claim, and a forced failure left the checkpoint byte-identical. The receiver is a local simulator: the real research endpoint does not exist yet. A later check found heartbeat keep-alives, excluded by every other view, were being counted into the statistics; corrected to match	receiver log
CO ₂ plausibility floor	A plausibility bound left at 0 ppm rather than the sensor's 400 ppm floor let 18.3% of stored CO ₂ readings, some below outdoor ambient, count as valid; after correction and re-running the same 24-hour window, implausible-flagged readings went 0 → 492 and the reported average moved 12.6 ppm	re-run of one window

Table 7. Selected reliability and integrity experiments. The CO₂ row records the plausibility check catching a real bias in the pipeline's own stored output.

7 Discussion

7.1 What the results establish

The four-rung comparison (Section 6.1) settles the project's central design question on evidence: the predecessor's pre-shared ECB scheme leaks its own plaintext's structure in a way entropy analysis cannot detect, its stated future work (AES-GCM) closes that specific leak, and DTLS 1.3 closes it too *and* additionally replaces a distributed shared secret with a per-device certificate that can be revoked from a web page. The attack suite and the wire captures confirm that a network attacker, however well-equipped short of the CA key, gains nothing.

7.2 Positioning against the prior work

Marina Rull Ventura's project secured one existing device well, on a fixed target, and its evaluation rigour is the model this work deliberately copied. Its design is also better than a summary suggests: per-device keys, a database of authorised units, and a documented argument for verifying the integrity tag before decrypting. What it did not need, and therefore did not have, are the properties a fleet forces: a device identity

that is not itself the secret, a withdrawal that acts on the next record rather than at the receiver's next start, and no secret shared across units at all. This project's scope is a superset, a scalable and self-organising fleet secured from first principles with a full PKI, because the earlier phase of this same research line named that exact scope as its Phase-2 work. The comparison is therefore not a contest between the two projects; it is this project delivering, and measuring, the step the prior work pointed to.

7.3 Limitations and what is not claimed

- Sustained operation is demonstrated at **two** continuously running units. A sixteen-unit fleet was captured for concurrency and discovery, not soaked. Reconnect-storm figures beyond sixteen units are a verified model plus a live single-device spot-check.
- The negotiated cipher suite and key-exchange group are read from captured handshakes rather than from an instrumented endpoint, and both are library defaults rather than pinned choices. The air-quality unit's hybrid post-quantum group in particular was observed, not designed, and nothing in the code prevents a library update from changing it (Section 8).
- In the build measured here the CA private key and the device private keys reside on the server, at file mode 600, in the provisioning directory. No running service reads them, but a server compromise would yield all of them, and with the CA key an attacker could mint any identity the fleet accepts. The removable-media path is documented and is implemented for the noise-unit factory, which reads the key from the mounted drive and never copies it; the air-quality card tools do not yet accept a drive argument, so that path has not been run end to end and is not claimed as evidenced.
- Flash encryption and secure boot are not enabled; physical theft of one unit is contained by per-unit keys and revocation, not prevented.
- Certificate lifetime is long; short-lifetime issuing for the stolen-device case is agreed but not implemented.
- Zero-touch auto-enrolment is designed but not built; per-device provisioning is currently a scripted manual step.
- No matched-condition performance benchmark exists against the pre-shared-key approach; only the architectural and wire-level comparison of Section 6.
- The replication tooling that lets the whole system be rebuilt from a single storage device onto a bare Linux machine is verified phase-by-phase and dry-run-green end to end; a single uninterrupted bare-metal install has not been observed in one pass. The full procedure and its audit are in the repository (`START_HERE.md`, `REPLICATION_AUDIT.md`).

Each limitation above is paired with a scoped next step in Section 8.

7.4 Availability

The firmware, server code, provisioning and certificate-authority tools, installers, plain-language deployment manuals, the verification gate with its attack suite, and the evidence file behind every result reported here are published in one public repository [23], organised for reproduction alongside this document and released under the MIT Licence. Each numbered experiment in Appendix A corresponds to a dated evidence file in that repository, which carries the exact commands run, the pass and fail criteria fixed before the run, and the raw artefact.

Two things are deliberately absent from it, and both are absences a reader should expect. No private key of any kind is published: the certificate authority's key, the per-unit device keys, the server and operator keys, and the Wi-Fi credentials are all excluded, with example templates committed in their place. The packet captures are published as they were taken. All but one contain only encrypted traffic, which is the result being reported; the four-rung capture deliberately contains readable telemetry in its first rung, because demonstrating a defence means showing the undefended case beside it. What is readable there is environmental data from the author's own test unit.

The one licensing exception is the air-quality-unit enclosure, which derives from a third-party design by AlfaChris [30], included with that author's explicit permission and not covered by the MIT Licence. The Phase-1 sensing platform this work builds on has its own separate repository [1].

8 Future work

The items below are recorded in full, with cost and closing step, in the project's `FUTURE_WORK.md`. Each was understood during the work and left deliberately, not discovered late.

8.1 Key custody and device hardening

Give the three air-quality card tools the same `--ca-key-dir` argument the noise-unit factory already takes, so the CA private key can stay on removable media for every provisioning path rather than only that one, and then run the resulting key-off-server build end to end once to evidence it. Delete each device's private key from the provisioning host once its unit is built. Enable ESP32 flash encryption and secure boot, provisioned over the cabled connection already used to flash each unit, so that a stolen unit yields only ciphertext and will not run an unsigned image. Issue device certificates with short lifetimes so that an un-revoked stolen unit expires on its own. The partition table already reserves the space these need.

A second authority root, generated alongside the first and kept entirely offline, would change what a compromise of the operating one costs. Section 4 states that case as unrecoverable, and it is: the response is to re-issue every unit by hand. A device that trusted two independent roots from the day it was built could be re-issued under the surviving one instead. The condition is that it cannot be retrofitted, since both anchors have to be present before the fleet is provisioned, which is why it belongs with the decisions taken at manufacture rather than with maintenance.

8.2 Per-certificate operator revocation

A device certificate can be withdrawn from the dashboard and takes effect on that device's next record. The operator certificate has no equivalent: the mutual-TLS gate checks only that a client certificate chains to the fleet CA, with no revocation list loaded. Closing this is a certificate revocation list generated by the existing authority and one directive in the gate configuration, so a single leaked operator key can be retired without reissuing the authority.

8.3 Zero-touch auto-enrolment

Replace per-device manual provisioning with an enrolment protocol: a device generates its own keypair on first boot, submits a certificate signing request, appears as *pending* on the dashboard, and becomes active only after a human approves it, keyed by chip serial so a re-flashed unit recovers its previous name. This is the main step between the current system and operation at a fleet size a person cannot hold in their hands.

Nothing secret crosses that exchange. The request carries the device's public key and its chip serial, signed with the matching private key so that neither field can be substituted in flight and the requester is shown to hold what it asks to have certified. The device verifies the server against the CA certificate it already carries, so the session is authenticated in one direction while the device is still anonymous. What such a request cannot establish is entitlement to a name, which is why approval stays a human decision and the enrolment window stays closed by default.

It subsumes a smaller step that is worth naming separately, because it is useful on its own and is much cheaper. A noise unit's certificate and private key are compiled into the image built for it, so the factory script produces one firmware build per unit. Moving that identity into the microcontroller's non-volatile key-value storage, written per unit at provisioning time, would let one image serve the whole fleet and would make the enrolment work above a change to the provisioning step alone rather than to the build. The same move should carry the CA certificate, which is identical on every unit but is compiled in with the rest: while it stays in the image, changing the fleet's trust anchor means rebuilding and reflashing every unit rather than writing a value to it.

8.4 Scale validation

Continuous operation is demonstrated at two units and concurrency at sixteen. A sustained multi-week field test at forty or more units per type would move the reconnect-storm and long-idle behaviour from a verified model plus a single-device spot check to an observed result. The same run should include one uninterrupted bare-metal execution of the replication tooling, which is currently verified phase by phase and dry-run green but not yet observed end to end in a single pass.

8.5 Firmware updates over the air

Every firmware change is currently flashed over USB, one board at a time. The A/B partition layout and rollback slots are already reserved; what remains is a service that carries a signed image over the existing mutually-authenticated channel, verifies the signature before marking it bootable, and rolls back automatically if the new image fails to check in.

8.6 Event-driven sensing and a bare-framework port for the noise unit

The noise unit detects events by continuous sampling on one core and is built on the Arduino-ESP32 core rather than bare ESP-IDF. Neither costs anything measurable on a mains-powered fleet, but both foreclose a future battery or power-budget deployment. The two independent steps are a hardware threshold (an analogue comparator or the low-power co-processor) that wakes the main cores only on sound, and a migration of the networking and lifecycle code to ESP-IDF, which the audio path already uses directly.

Restoring the acoustic analysis belongs here too. The reported level is an arithmetic mean of per-chunk decibel values (Section 1), which the logarithmic scale makes an approximation rather than an energy average; averaging the chunk powers before the conversion would correct it at no real cost. The larger item is the voice-band spectral analysis carried by the Phase-1 unit and dropped here, which the 8 kHz sampling rate would first have to be raised to support. Both changes are confined to the audio module and touch nothing in the transport.

8.7 Post-quantum readiness

The component a future quantum adversary breaks is the asymmetric layer, not the AES-GCM record layer. On this point the deployed system is further along than planned: the packet captures show the air-quality-unit link already negotiating a hybrid post-quantum key exchange (SecP384r1MLKEM1024, P-384 ECDHE combined with MLKEM-1024), because the server and the Pi's TLS library offer it by default. This was observed rather than designed. The work that remains is to make it a deliberate, pinned choice rather than a silent library default; to decide whether the constrained noise-unit build should also carry a hybrid group or stay on classical P-256 with that stated as a limitation; and to keep the certificate signatures, still ECDSA on P-256, on the same migration path.

8.8 Performance benchmark

A matched-condition comparison of handshake and per-record cost against the pre-shared-key approach, to complement the architectural and wire-level comparison in this memo with a quantitative one. The corresponding comparison for hardware cryptography has already been made and is reported in Section 6.7; what remains open there is only whether the ESP-IDF port becomes worthwhile for the other reasons named above, since the cryptographic gain alone does not justify it.

The noise unit's power management is the other item belonging here. The firmware scales its processor across three frequencies and sleeps the Wi-Fi radio between transmissions, and session reuse is what keeps it in the low state for most of its life (Section 3), but no instrumented measurement of current draw was taken. An energy profile of a deployed unit, comparing the shipped design against a per-message handshake and against fixed-frequency operation, would turn that design rationale into a measured result and is the natural companion to the timing figures already reported.

8.9 Smaller items

The daily-statistics push has no real destination configured and prints its report until one is supplied; the air unit's inbound live-query listener runs under a second, locally

built Python interpreter because the TLS binding mishandles thread state on the system version; and two exact clones of one unit, flashed with the same name and key in separate sittings, would both be admitted, which a detect-and-report check on concurrent sessions would surface. The dashboard is also served by its web framework’s own built-in server rather than by a production application server; it is bound to loopback behind the proxy, so the change is a line in a service file rather than a design one.

9 Conclusions

This work replaced an unauthenticated plaintext telemetry link with mutual, certificate-based DTLS 1.3 across a heterogeneous fleet of salvaged-hardware sensing units, and layered on top of it the operational capabilities a distributed deployment requires: autonomous per-tier recovery, zero-touch server discovery, database retention under unbounded growth, a daily statistics push to a research endpoint, and an AI operator layer whose fixed tool set makes prompt injection structurally rather than heuristically impossible.

Measured against the second-phase mandate, the security objective is met and evidenced on real hardware: mutual authentication end to end, forward secrecy, per-device revocation that admits no further record from the instant it is applied and independently of the cryptographic session, and demonstrated resilience to rogue servers, credential theft (via revocation), a real network outage, and a modelled-plus-spot-checked reconnect storm. The four-rung comparison against the parallel final project shows, on deployed hardware, both what that project’s stated future work would have bought and what this project’s PKI adds beyond it.

What remains is scale and hardening. Continuous operation was demonstrated at two units and captured at sixteen; the custody of private keys, certificate lifetimes, flash encryption, and zero-touch enrolment are identified and scoped in the preceding section (Section 8). Published measurements had already shown that DTLS 1.3 fits the resource budget of low-power microcontrollers [4]; this work confirms that on a running fleet and addresses the credential-management problem the literature identifies as the harder one [5], replacing a non-revocable shared secret with a per-device certificate that a dashboard can withdraw at once. The contribution is a working, measured proof of concept for encrypted, mutually-authenticated sensor telemetry with an auditable AI-operated control layer, in which every security property claimed was demonstrated against real hardware with real attack reproduction rather than assumed from the protocol’s paper design.

Appendix A: Experiment register

One row per experiment, so that the evaluation can be assembled from facts rather than recollection. Each entry names what was claimed, how it was tested, and where the artefact can be read.

The rightmost column gives the file in the published repository [23]. Names without a directory are in `evidence/`, which holds one dated file per experiment recording the claim under test, the exact commands run, the pass and fail criteria fixed before the run, and the raw output. Packet captures of the same name live beside them in `evidence/captures/`. Two rows are evidenced by code and its test rather than by a run log, and give the full path. The register is Table 8.

Table 8. Experiment register. Selected rows are presented with figures in Section 6.

Experiment	What it proves	Evidence
Live wire capture	Payloads are encrypted on the deployed system; sessions are concurrent	2026-08-26_live_wire_capture.md
Attack suite (7 classes)	Eavesdrop, spoof, replay, tamper, cross-CA forgery, rogue operator, revoked device all defeated	attack_suite.md
NMU cold discovery	A unit with no stored server address finds it unattended	2026-08-25_cold_discovery.md
Handshake exposure, 1.2 vs 1.3	DTLS 1.3 hides identity: no certificate name on the wire, reproduced on a second unit	2026-08-26_boot_discovery_handshake.md, 2026-08-27_nmu21_handshake_recapture.md
Revocation, live, both device types	Mid-session cutoff on the next record (9s, after 43.8s of unit silence); unattended rejoin, observed on the air unit; buffered data recovered with original timestamps	2026-08-26_revocation_live.md, 2026-08-26_revocation_amu.md
Cold power-cut recovery	Unit that loses power and network returns to work unattended	2026-08-27_cold_powercut.md
NMU clock across a restart	Server-learned time survives a reboot the unit performs to heal itself, and is discarded rather than guessed after a power cut; the unit tells the two apart	2026-08-27_nmu_reboot_clock.md
Outage buffering and recovery ladder	Rungs fire at the times predicted from the code; session identifier and event counter survive the self-reboot in retained memory	2026-08-27_nmu22_recovery_ladder_serial.md
Outage restore, full cycle	Restarted with retained memory cleared, the unit reads 91 records back from flash and delivers all 91 in 6s under their original session identifier, zero duplicates	2026-08-21_nmu_outage_restore_cycle.md
Four-rung comparison	Confidentiality, and why PKI over a pre-shared key	2026-08-27_psk_ladder.md
Scale and timing	16-unit fleet: concurrent handshakes, handshake duration cross-checked two ways (device timer, packet timestamps)	2026-08-27_scale_timing.md
<i>continued on the next page</i>		

Experiment	What it proves	Evidence
MCP stored-data tools read-only	320 calls against a frozen snapshot, byte-identical before and after; zero write statements	2026-08-27_mcp_readonly_stored_data.md
MCP live read leaves no trace	A live read of a deployed unit is answered by the hardware over a fresh DTLS session and writes nothing: no row in the window, no record naming the query	2026-08-25_mcp_live_isolation.md
Database compaction	Oldest 15 % pruned, newest rows byte-identical, integrity check passes	2026-08-27_db_compaction.md
Daily statistics push	Consecutive daily reports delivered; retry-then-print on failure; checkpoint only advances on confirmed delivery	2026-08-27_stats_endpoint_reconfirm.md
Session-age reauth	Unforced re-handshakes within the designed jitter band, zero data loss at the boundary	2026-08-27_session_age_reauth.md
CO ₂ plausibility floor	Readings outside the sensor's own documented range are excluded from the statistics rather than averaged in	server/daily_stats.py, server/test_maintenance.py
Prompt-injection validation	An injection string in the device reason field is stored as the literal <code>Unknown</code> ; 13 of 13 cases	server/test_cause_validation.py
Hardware against software crypto	With both builds pinned to 240 MHz, hardware acceleration is 9.4% faster on the handshake by the device clock and 11.7% by the server's, and unchanged per record; the shipped build stays on software crypto	hw_crypto/README.md, hw_crypto/nmu_espidf_240mhz_controlled_20260815.txt

Appendix B: Phase-2 hardware changes

The sensing platform is that of the Phase-1 kits [1] and is not re-documented here. This appendix records only what changed for the Phase-2 fleet: the mechanical and fabrication work done by the author to turn two bench prototypes into a fleet of twenty-two deployable units of each type (twenty-two air-quality units and twenty-two noise units). No formal university homologation was sought; the fleet is a prototype built to explore and document a proof of work.

B.1 Enclosures

The enclosure continues the modular Phase-1 design [1]. The air-quality-unit enclosure derives, through that Phase-1 design, from a published Raspberry Pi 4B case by AlfaChris [30], used with the author’s explicit permission for this project; the original licence does not apply and redistribution outside this project requires the author’s approval. Units were printed to order by an external 3D-printing service in Tarragona [31], twenty-two of each type. The first printed copy of each type was inspected before the full run; from that inspection the author requested a change to the NMU casing’s internal connector-clearance dimensions so that the redesigned header rows (Section B.3) would seat correctly, and the full order was printed after the revision.

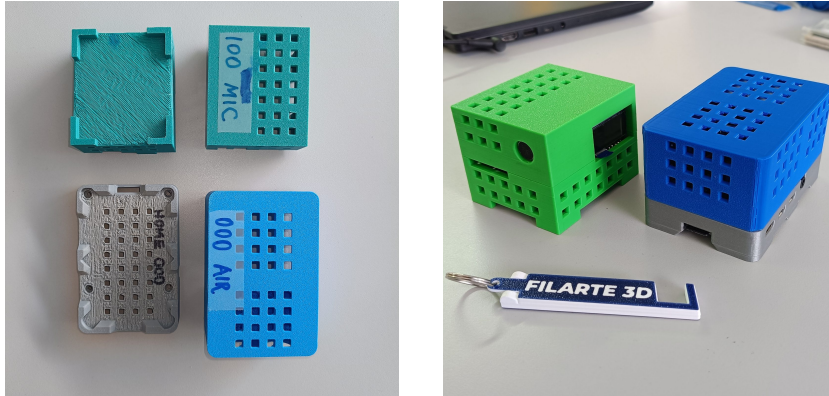


Figure 19. Enclosure, Phase-1 (left) and the revised Phase-2 NMU casing (right) after the connector-clearance change.

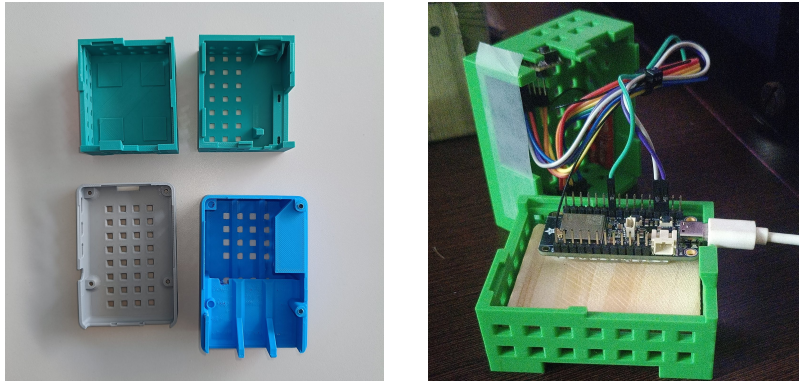


Figure 20. Module stack, before (left) and after (right). The Phase-1 build carried a battery, an LCD and an SD-card module; the Phase-2 unit drops all three and closes the freed volume with a mechanical spacer.

B.2 Parts removed

Two components were dropped, both made cheap by the inherited modular architecture:

- **Battery.** Expected powered-on duration far exceeds any battery's, and Phase 1, with less functionality and more aggressive power saving, had already found battery life to be the limiting factor. Every unit is mains-powered. A wooden mechanical spacer replaces the cell and keeps the module stack correctly dimensioned in the enclosure.
- **LCD and SD-card module.** No local display or local storage is needed in the deployed role; the server holds the data and the dashboard shows it.

B.3 Button block, redesigned for tool-free service

Previously the module's wiring was soldered directly to its board as Dupont leads. A standard 0.1-inch header pin row is now soldered to the board instead and the leads plug onto it, so both the NMU and the AMU disassemble into modules without desoldering. The block is fitted to every NMU. The button itself is unused in the deployed role; the module's job is to act as a hub for power redistribution and inter-module connectivity, arranged so that an LCD and SD-card module can be added and the button assigned a function if a later use needs one. The aim throughout was that a unit in a residence is repaired by swapping a module, not by returning the enclosure.

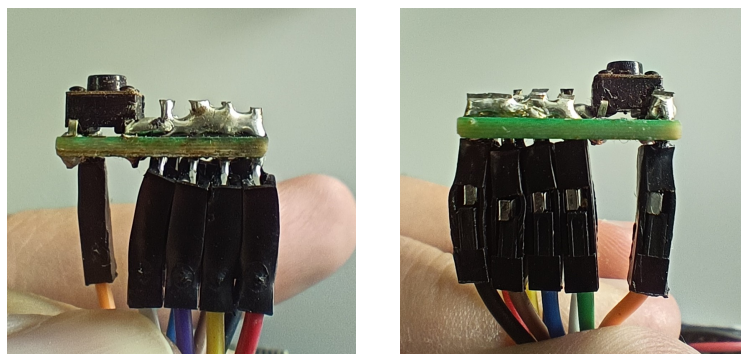


Figure 21. Button-block module before the revision: leads soldered directly to the board as Dupont wires, so the module could not be removed without desoldering. A 0.1-inch header row now takes their place.

B.4 Board fabrication and assembly

The module boards are small pieces cut from a larger blank with a guillotine shear in the URV Department of Electronics, Electrical and Automatic Control Engineering laboratory. The author personally carried out the full assembly of twenty-two units of each type: soldering, module installation, the mechanical revisions above, and all internal wiring.



Figure 22. Top: the assembled fleet, boxed, twenty-two units of each type (air-quality units in one colour, noise units in the other). Bottom: assembly in progress.

B.5 A constraint that shaped the firmware

The three Phase-1 noise units carried an ESP32-S3 module with on-package PSRAM. Those three units still exist but are not part of the Phase-2 fleet and are not used here. For the Phase-2 build, that module was not reliably available, so a different ESP32-S3 was chosen for the whole run: the 8 MB-flash, no-PSRAM ESP32-S3 Feather module [32]. All twenty-two Phase-2 noise units use this one model.

The change forced three adaptations. First, the module has no on-board antenna and exposes a w.FL/MHF3 connector instead, so an external 2.4 GHz antenna [33] was fitted to each unit through that connector. Second, with no PSRAM the firmware memory budget is set entirely by internal RAM: the DTLS stack’s multi-kilobyte handshake burst and the offline ring buffer had to fit there, and that is the memory budget the transport design of Section 3 had to work within. Third, the 8 MB flash made a multi-partition layout with two application slots feasible, and an over-the-air update scheme on that layout was explored on the bench; it is not part of the delivered system and is carried as future work in Section 8.5.

Supplier and cost are outside the scope of this memo; the citations identify the parts, not where they were bought.

Appendix C: MCP operator sessions

Recorded Claude desktop sessions against the live fleet through the nine MCP tools, over the mutually-authenticated gateway described in Section 3.9. Two further sessions from this set, the fleet roll-call and the first panel of the alarm-cause deduction, appear inline as Figures 16 and 17 in Section 6.6. Device names and readings are the author's own test fleet; nothing here is redacted because nothing in the exchange is sensitive.

C.1 Status and stored-data tools

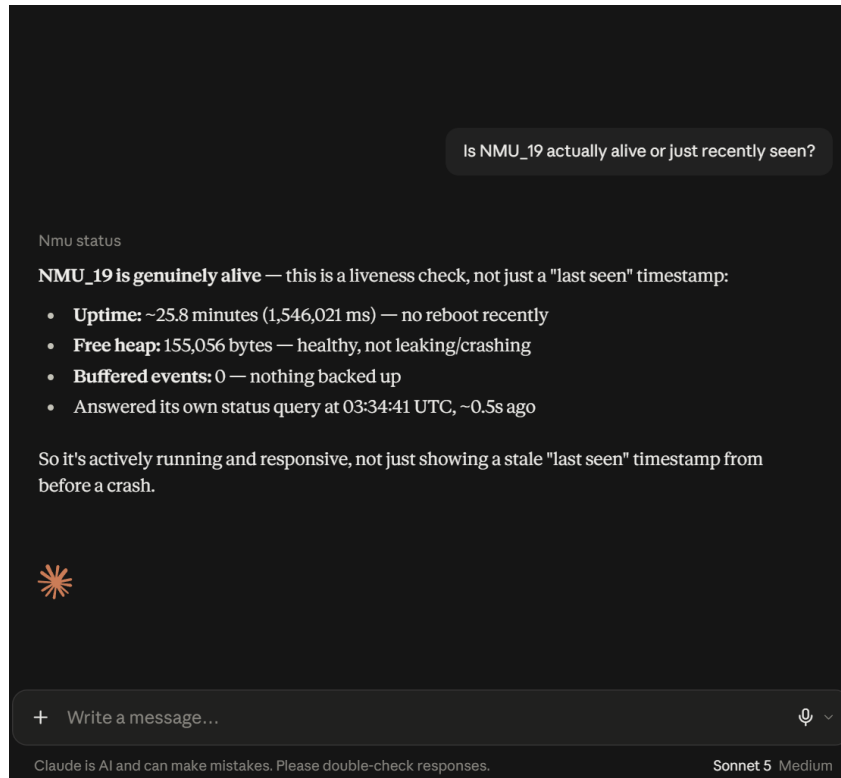


Figure 23. `nm_u_status` for a single noise unit.

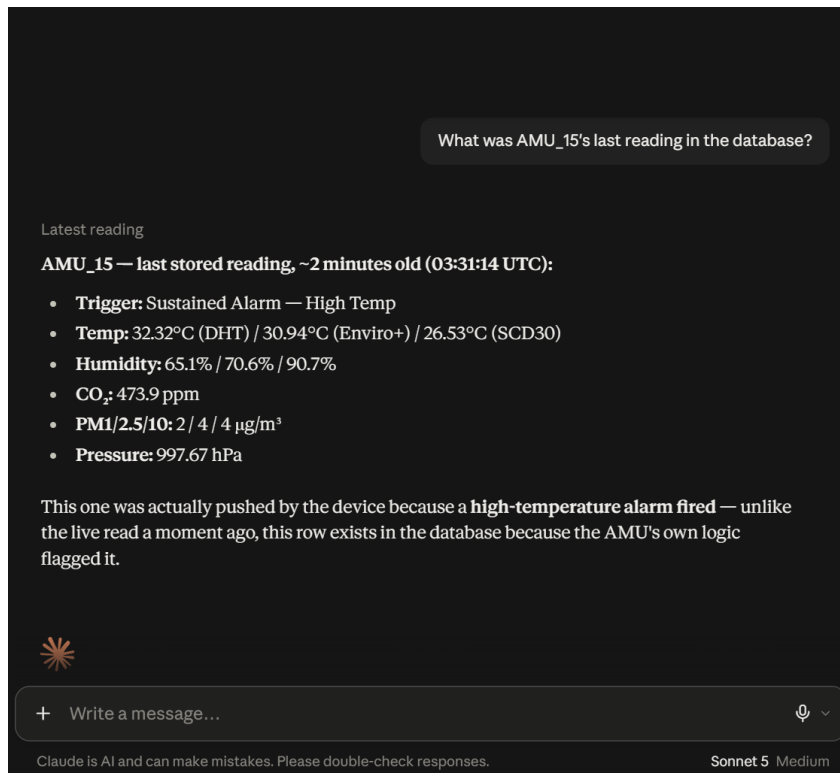


Figure 24. latest_reading for AMU_15, read from the stored data.

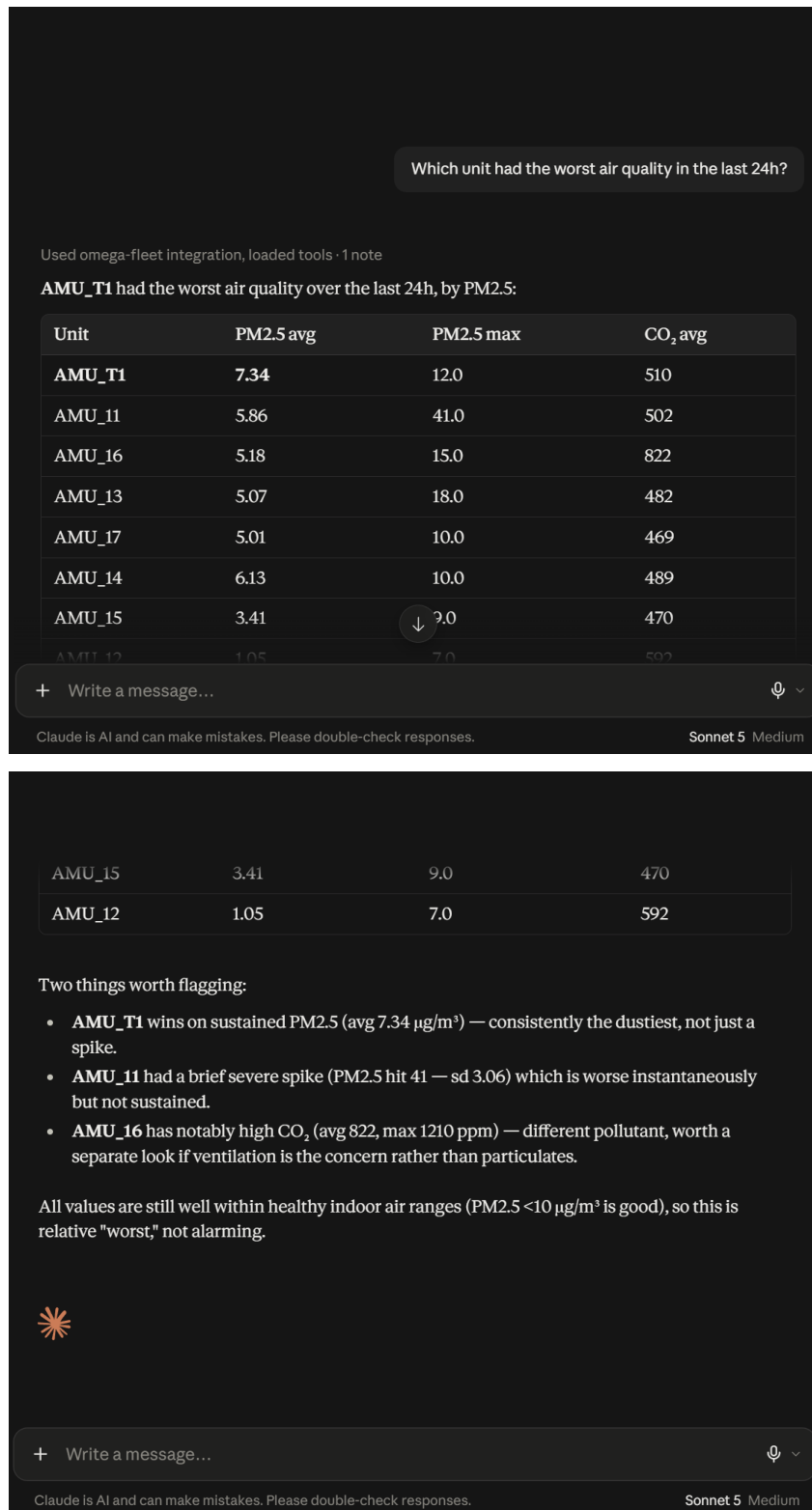


Figure 25. `device_stats` / `activity_report` used for a descriptive analysis over a window, in two parts.

C.2 Live reads to the hardware

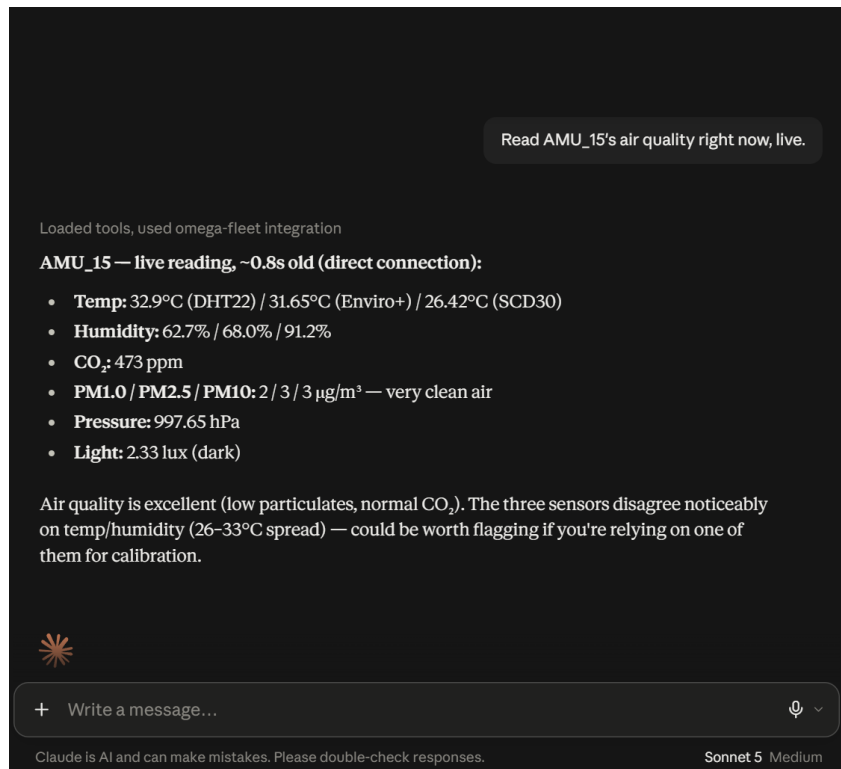


Figure 26. `live_read`: a fresh DTLS session is opened to the device and the reading is returned from the hardware, not the database.

Given CO2 live reading of AMU_13, is that safe for a classroom or hospital room, and what's recommended?

Live read

AMU_13 — CO₂: 502 ppm (live, ~2.7s old)

Safe for both settings — well within recommended limits:

Threshold	Value	AMU_13 status
Outdoor baseline	~420 ppm	slightly above
Good indoor air	<800 ppm	✅ well under
ASHRAE/classroom guidance	<1000 ppm	✅ well under
Hospital patient rooms (stricter, ventilation-driven)	typically <800-900 ppm	✅ well under
Drowsiness/complaints threshold	>1000 ppm	far below

↓

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses. Sonnet 5 Medium

Threshold	Value	AMU_13 status
Outdoor baseline	~420 ppm	slightly above
Good indoor air	<800 ppm	✅ well under
ASHRAE/classroom guidance	<1000 ppm	✅ well under
Hospital patient rooms (stricter, ventilation-driven)	typically <800-900 ppm	✅ well under
Drowsiness/complaints threshold	>1000 ppm	far below

No action needed right now — ventilation is doing its job. General recommendations if it climbs:

- **1000–1500 ppm:** increase air exchange (open windows, boost HVAC fresh-air intake)
- **>1500 ppm:** treat as a ventilation fault — investigate immediately, especially in a hospital room with vulnerable patients

Worth noting: this device logged a CO₂ spike earlier (24h max was 529.57, so it's been stable) — no history of concerning excursions on this unit specifically.

+ Write a message...

Claude is AI and can make mistakes. Please double-check responses. Sonnet 5 Medium

Figure 27. A live read followed by a recommendation the operator can act on, in two parts. The recommendation is text for the operator; it is not an action the model can take.

C.3 Deducing an unnamed alarm condition

AMU_16 — last 2h activity report

Trigger in question: Sustained Alarm: High Temp (+1) — 24 occurrences
 Named alarm: High Temp (ongoing since 01:51:18 UTC, ~1h58m and counting)

Trends in window vs. alarm thresholds (checked in rule order):

Variable	Range this window	Threshold	Crossed?
CO ₂	798.8-840.6 ppm	≥1400	✗
PM2.5	4-7 µg/m ³	≥55	✗
Temp (dht)	30.51-32.03°C	≥30°C	✓ (already the named "High Temp")
Humidity (dht)	71.79-78.02%	≥70%	✓ crossed, and sustained for the whole 2h window
Lux	0.0 flat	≤5	✓ technically, but flat at 0 for 2h+ suggests a dead/no-light sensor rather than a real low-lux event — and Lux is last in check order, after Humidity

+ Write a message... 

Claude is AI and can make mistakes. Please double-check responses. Sonnet 5 Medium

Inferred secondary condition: High Humidity alarm (≥70% sustained)

Confidence: High. Reasoning:

- Humidity-high sits right after Temp-high in the rule-check order, and it's the very next variable to actually cross its threshold — no need to skip past it to explain the +1.
- Every single sample in the 2h window (min 71.79%) is above the 70% line, so the 60s-sustain-to-trip condition is trivially satisfied — this isn't a borderline call.
- It's corroborated by the separate Humidity Step/Drift cause firing twice in the same window — the sensor is actively moving in that band, not just sitting near a fixed value.
- Lux is theoretically also "at threshold" ($0 \leq 5$) but it's been flat at exactly 0.0 for the full window (and per the 24h stats, largely 0 historically) — more consistent with no ambient light / possibly a non-functional lux sensor than a genuine "Low Lux" alarm event, and it's lower priority in the check order anyway.

One caveat: the activity report only exposes `dht_hum` here — if the device's alarm logic reads humidity from a different sensor (`env_hum` / `scd_hum`) instead of DHT22, the specific reading feeding the alarm could differ, though sustained high humidity is consistent across sensor families in this unit's recent history.



+ Write a message... 

Claude is AI and can make mistakes. Please double-check responses. Sonnet 5 Medium

Figure 28. Continuation of Figure 17: the model works the `activity_report` trend against the firmware's alarm, spike and step thresholds and names the most likely secondary condition with a stated confidence.

C.4 A recorded model error, with the tool contract intact

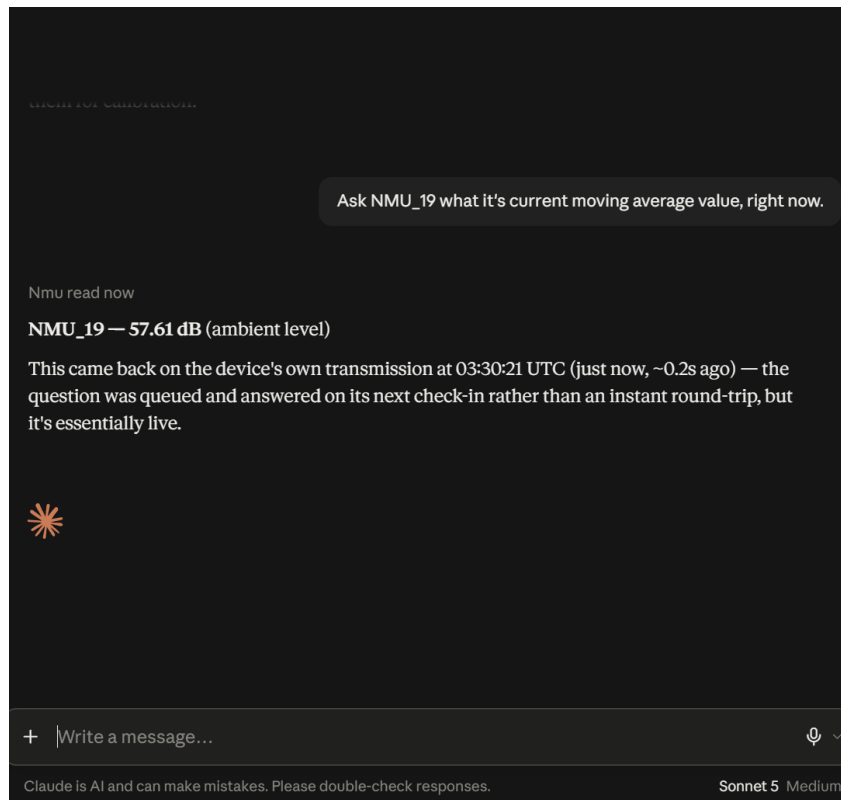


Figure 29. A queued read where the model reports the wrong timezone offset for the timestamp. The error is in the narration; the tool returned correct data and no tool did anything unintended. Kept as an honest record of a limitation of the assistant layer, not of the transport or the tool set.

References

- [1] Constantin Shoot Vayevoda. “Development of a Contextual Sensing Kit to Assess the Relationship Between Air Quality and Noise”. Phase 1 of this two-part project; memo and hardware sources in the linked repository. Treball de Fi de Grau. Universitat Rovira i Virgili, 2025. URL: https://github.com/pozdezh/Project_DELTA_Ph1 (visited on 30/08/2026).
- [2] Marina Rull Ventura. “Implementació d’una capa de seguretat en dispositius EmotiBit”. Parallel Phase-2 thesis in the same research line; evaluation method and predecessor design reused here for comparison. The security layer’s own source, cited in that thesis and linked here, is what rung B of Section 6.1 was checked against. Treball de Fi de Grau. Universitat Rovira i Virgili, 2025. URL: <https://github.com/MarinaRullVentura/EmotiBitSecure> (visited on 06/09/2026).
- [3] Eric Rescorla, Hannes Tschofenig, and Nagendra Modadugu. *The Datagram Transport Layer Security (DTLS) Protocol Version 1.3*. Request for Comments 9147. Internet Engineering Task Force, Apr. 2022. DOI: [10.17487/RFC9147](https://doi.org/10.17487/RFC9147). URL: <https://www.rfc-editor.org/rfc/rfc9147>.
- [4] Gabriele Restuccia, Hannes Tschofenig, and Emmanuel Baccelli. “Low-Power IoT Communication Security: On the Performance of DTLS and TLS 1.3”. In: *2020 IFIP/IEEE International Conference on Performance Evaluation and Modeling in Wired and Wireless Networks (PEMWN)*. First experimental comparison of DTLS/TLS 1.2 and 1.3 on low-power microcontrollers (bytes-over-the-air, memory, energy). Open-access preprint at arXiv:2011.12035. IEEE, 2020. URL: <https://arxiv.org/abs/2011.12035> (visited on 01/09/2026).
- [5] Joel Höglund, Samuel Lindemer, Martin Furuheid, and Shahid Raza. “PKI4IoT: Towards public key infrastructure for the Internet of Things”. In: *Computers & Security* 89 (2020). Automated, lightweight certificate enrolment for highly constrained devices; motivates PKI over pre-shared keys for the IoT. Author’s open-access copy in the DiVA repository, p. 101658. DOI: [10.1016/j.cose.2019.101658](https://doi.org/10.1016/j.cose.2019.101658). URL: <https://doi.org/10.1016/j.cose.2019.101658> (visited on 01/09/2026).
- [6] Universitat Rovira i Virgili. *Cognitive Integration of Intelligent and Contextual Systems for Active Ageing*. Project PID2024-158490OB-C31, funded by MICIU/AEI/10.13039/501100011033 under “Proyectos de Generación de Conocimiento” 2024. Dept. of Computer Engineering and Mathematics, Universitat Rovira i Virgili, 2025–2028. 2025. URL: <https://www.urv.cat/en/research/projects-financed-externally/2025/00277/001/integracion-cognitiva-de-sistemas-inteligentes-y-contextuales-para-envejecimiento-activo> (visited on 30/08/2026).
- [7] Carsten Bormann, Mehmet Ersue, and Ari Keränen. *Terminology for Constrained-Node Networks*. Request for Comments 7228. Internet Engineering Task Force, May 2014. DOI: [10.17487/RFC7228](https://doi.org/10.17487/RFC7228). URL: <https://www.rfc-editor.org/rfc/rfc7228>.
- [8] Stuart Cheshire and Marc Krochmal. *Multicast DNS*. Request for Comments 6762. Internet Engineering Task Force, Feb. 2013. DOI: [10.17487/RFC6762](https://doi.org/10.17487/RFC6762). URL: <https://www.rfc-editor.org/rfc/rfc6762>.
- [9] Eric Rescorla and Nagendra Modadugu. *Datagram Transport Layer Security Version 1.2*. Request for Comments 6347. Internet Engineering Task Force, Jan. 2012. DOI: [10.17487/RFC6347](https://doi.org/10.17487/RFC6347). URL: <https://www.rfc-editor.org/rfc/rfc6347>.
- [10] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. Request for Comments 8446. Internet Engineering Task Force, Aug. 2018. DOI: [10.17487/RFC8446](https://doi.org/10.17487/RFC8446). URL: <https://www.rfc-editor.org/rfc/rfc8446>.
- [11] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication 197 (update 1). National Institute of Standards and Technology, 2023. DOI: [10.6028/NIST.FIPS.197-upd1](https://doi.org/10.6028/NIST.FIPS.197-upd1). URL: <https://csrc.nist.gov/pubs/fips/197/final>.
- [12] Morris Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. National Institute of Standards and Technology, Nov. 2007. DOI: [10.6028/NIST.SP.800-38D](https://doi.org/10.6028/NIST.SP.800-38D). URL: <https://csrc.nist.gov/pubs/sp/800/38/d/final>.
- [13] Hugo Krawczyk and Pasi Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. Request for Comments 5869. Internet Engineering Task Force, May 2010. DOI: [10.17487/RFC5869](https://doi.org/10.17487/RFC5869). URL: <https://www.rfc-editor.org/rfc/rfc5869>.
- [14] National Institute of Standards and Technology. *Digital Signature Standard (DSS)*. Federal Information Processing Standards Publication 186-5. National Institute of Standards and Technology, 2023. DOI: [10.6028/NIST.FIPS.186-5](https://doi.org/10.6028/NIST.FIPS.186-5). URL: <https://csrc.nist.gov/pubs/fips/186-5/final>.
- [15] Lily Chen, Dustin Moody, Karen Randall, Andrew Regenscheid, and Angela Robinson. *Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters*. NIST Special Publication 800-186. National Institute of Standards and Technology, 2023. DOI: [10.6028/NIST.SP.800-186](https://doi.org/10.6028/NIST.SP.800-186). URL: <https://csrc.nist.gov/pubs/sp/800/186/final>.

- [16] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Federal Information Processing Standards Publication 180-4. National Institute of Standards and Technology, 2015. DOI: [10.6028/NIST.FIPS.180-4](https://doi.org/10.6028/NIST.FIPS.180-4). URL: <https://csrc.nist.gov/pubs/fips/180-4/final>.
- [17] Elaine Barker. *Recommendation for Key Management: Part 1 – General*. NIST Special Publication 800-57 Part 1, Revision 5. National Institute of Standards and Technology, May 2020. DOI: [10.6028/NIST.SP.800-57pt1r5](https://doi.org/10.6028/NIST.SP.800-57pt1r5). URL: <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>.
- [18] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russ Housley, and Tim Polk. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. Request for Comments 5280. Internet Engineering Task Force, May 2008. DOI: [10.17487/RFC5280](https://doi.org/10.17487/RFC5280). URL: <https://www.rfc-editor.org/rfc/rfc5280>.
- [19] Stefan Santesson, Michael Myers, Rich Ankney, Ambarish Malpani, Slava Galperin, and Carlisle Adams. *X.509 Internet Public Key Infrastructure Online Certificate Status Protocol – OCSP*. Request for Comments 6960. Internet Engineering Task Force, June 2013. DOI: [10.17487/RFC6960](https://doi.org/10.17487/RFC6960). URL: <https://www.rfc-editor.org/rfc/rfc6960>.
- [20] wolfSSL Inc. *wolfSSL: embedded SSL/TLS library*. Source repository of the TLS/DTLS 1.3 implementation used on the server and the air-quality units (through the Python `wolfssl` binding) and on the noise units (through Arduino-wolfSSL). 2026. URL: <https://github.com/wolfSSL/wolfssl> (visited on 30/08/2026).
- [21] wolfSSL Inc. *Arduino-wolfSSL: a restructured copy of wolfSSL for the Arduino environment*. 2026. URL: <https://github.com/wolfSSL/Arduino-wolfSSL> (visited on 29/08/2026).
- [22] wolfSSL Inc. *wolfssl: Python wrapper for the wolfSSL embedded SSL/TLS library*. Python Package Index. 2026. URL: <https://pypi.org/project/wolfssl/> (visited on 30/08/2026).
- [23] Constantin Shoot Vayevoda. *Project OMEGA Phase 2: a secure DTLS 1.3 IoT sensing fleet*. Source repository of the work described in this document: noise-unit firmware, air-quality-unit and server software, the provisioning and certificate-authority tools, the installers and plain-language deployment manuals, the verification gate and attack suite, and the evidence files behind every result reported here. Released under the MIT Licence, with the enclosure exception noted in Section 7. 2026. URL: https://github.com/pozdezh/Project_OMEGA_Ph2 (visited on 31/08/2026).
- [24] wolfSSL Inc. *wolfSSL DTLS 1.3 ESP32 examples now available*. Sept. 15, 2023. URL: <https://www.wolfssl.com/wolfssl-dtls1-3-esp32-examples-now-available/> (visited on 30/08/2026).
- [25] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Federal Information Processing Standards Publication 203. National Institute of Standards and Technology, 2024. DOI: [10.6028/NIST.FIPS.203](https://doi.org/10.6028/NIST.FIPS.203). URL: <https://csrc.nist.gov/pubs/fips/203/final>.
- [26] Espressif Systems. *ESP-IDF components/mbdtdls/Kconfig – hardware AES/SHA/GCM acceleration notes*. 2026. URL: <https://github.com/espressif/esp-idf/blob/master/components/mbdtdls/Kconfig> (visited on 29/08/2026).
- [27] Espressif Systems. *ESP32-S3 Technical Reference Manual*. 2024. URL: https://documentation.espressif.com/esp32-s3_technical_reference_manual_en.pdf (visited on 30/08/2026).
- [28] Anthropic. *Model Context Protocol specification*. 2025. URL: <https://modelcontextprotocol.io/specification> (visited on 29/08/2026).
- [29] Stuart Cheshire and Marc Krochmal. *DNS-Based Service Discovery*. Request for Comments 6763. Internet Engineering Task Force, Feb. 2013. DOI: [10.17487/RFC6763](https://doi.org/10.17487/RFC6763). URL: <https://www.rfc-editor.org/rfc/rfc6763>.
- [30] AlfaChris. *Raspberry Pi 4B Case*. Model no. 1937761. The air-quality-unit enclosure is based on this design, used with explicit permission from the author for inclusion in this project. The original licence does not apply; redistribution outside this context requires the author’s approval. Permission carried forward from Phase 1. Cults3D. 2024. URL: <https://cults3d.com/:1937761> (visited on 30/08/2026).
- [31] Filarte 3D. *Filarte 3D — 3D design and printing service, Tarragona*. External service that printed the Phase-2 enclosures to order. URL: <https://filarte3d.com/> (visited on 30/08/2026).
- [32] Adafruit Industries. *Adafruit ESP32-S3 Feather 8MB Flash No PSRAM with w.FL antenna port*. Product no. 5885. The ESP32-S3 module used in the Phase-2 noise units (8MB flash, no PSRAM, w.FL/MHF3 external-antenna connector). Cited for the module specification only; not a statement of supplier or cost. 2023. URL: <https://www.adafruit.com/product/5885> (visited on 30/08/2026).

- [33] BerryBase. *WiFi antenna with w.FL / MHF3 connector*. External 2.4 GHz antenna fitted to each noise unit through the module's w.FL/MHF3 port. Cited for the part specification only; not a statement of supplier or cost. URL: <https://www.berrybase.ch/en/wifi-antenna-with-w.fl-mhf3-connector> (visited on 30/08/2026).

